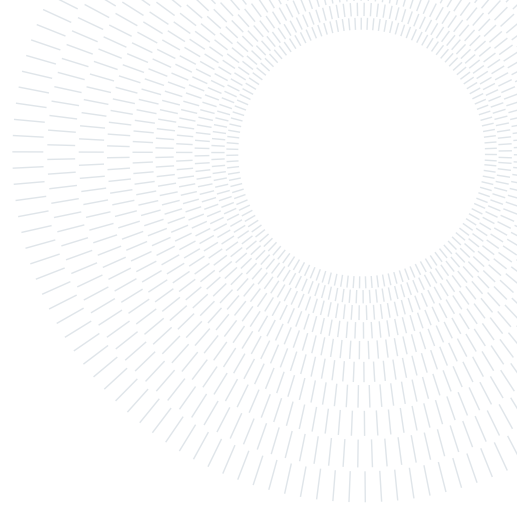




POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE



A Data-Driven Multi-Armed Bandit Framework for Adaptive Game-Based Handwriting Training in First-Grade Students

TESI DI LAUREA MAGISTRALE IN
BIOMEDICAL ENGINEERING - INGEGNERIA BIOMEDICA

Ilaria Moretti, 10713475

Advisor:
Prof. Simona Ferrante

Co-advisors:
Chiara Piazzalunga
Francesco Trovò

Academic year:
2024-2025

Abstract: Writing is an essential skill, and technology can be a fundamental tool for supporting students with learning difficulties. In particular, serious games can offer a system that allows for fun practice, avoiding boredom and frustration often associated with repetitive training exercises. This thesis proposes a Multi-Armed Bandit (MAB) framework capable of adapting difficulty in real-time based on player performance while sustaining high participation in the game "Number Tracing", a tablet-based serious game dedicated to handwriting played using a stylus. The game was modeled as a constrained MAB problem, in which each level represents an arm and a minimum constraint is imposed on the level of engagement. The reward was defined as tracing accuracy, measured as the percentage of the trace within the target boundaries, while engagement was operationalized through in-air time, capturing the player's active involvement during the task. In the experimental setup, a static player model was assumed. Several MAB algorithms, including epsilon-greedy, Upper Confidence Bound, and Thompson Sampling, were evaluated, along with different operative configuration, and tested on data gathered across a three-month training protocol on 66 first-grade children. The results showed that Thompson Sampling, on aggregated data, achieved the best performance in terms of cumulative reward and regret. Since the static assumption is a strong simplification of reality, a dynamic model was subsequently developed, considering a single operative configuration, including the contextual Linear UCB algorithm, which showed the best performance. The results obtained highlight the potential of the proposed frameworks, while underscoring the need for further studies on larger datasets and for evaluating the effectiveness in a real-world context.

Key-words: constrained Multi-Armed Bandit, constrained contextual bandit, serious game, children, handwriting, training

1. Introduction

In recent years, a growing trend toward the replacement of handwriting by digital alternatives has been observed [1]. Although these different instruments have diversified how writing is produced, handwriting remains a very important tool for a person's education. Indeed, writing is a core skill in educational systems: academic goals depend largely on language proficiency. Knowing how to write correctly is related to the ability to plan,

structure, and rework complex content: all activities that form the basis of academic assessments and success [2, 3]. However, acquiring this skill can often be problematic. Handwriting results from complex tasks involving linguistic, cognitive, and motor processes. In fact, the process that leads to the acquisition of writing ability follows a progression in early childhood. Children begin learning to write in kindergarten and the first year of primary school, with gradual improvements over the following year. During third grade, automaticity begins to develop as typical writing movements become rote responses. Handwriting continues to improve, evolve, speed up, refine, and become less variable during the later years of elementary school [3, 4].

Handwriting acquisition may not occur uniformly among children, who may experience different types of struggle. For some students, these difficulties may be caused by a Specific Learning Disorder (SLD), a condition that causes persistent difficulties in reading (dyslexia), writing (dysgraphia), or mathematics (dyscalculia) [5]. SLDs can manifest despite normal intellectual ability, and although they can be variable, there are patterns that can help healthcare providers identify them. With particular reference to the writing domain and so to dysgraphia, this often leads to: letter illegibility, insufficient letter spacing, slow writing, spelling difficulties, problems with syntax and composition, and fine motor coordination [4].

In other cases, difficulties in learning may be temporary and can be resolved over time with appropriate support [6]. Training protocols can vastly benefit both children with learning disorders and children who are simply experiencing delays [7, 8]. However, intervention should be prompt and adequate, as temporary delays can calcify into permanent difficulties if left unattended, while children with SLDs often go unrecognized due to long waiting lists, lack of objective data, and insufficient resources - facing years of struggle that breed discouragement, frustration, and low self-esteem [9]. Dysgraphia, specifically, cannot be diagnosed before third grade, when handwriting is expected to be mastered [10], thus missing crucial early years that could have been fundamental in strengthening the impaired ability.

Schools serve as crucial early warning and targeted training systems for learning difficulties and developmental issues. In Italy, Law 170/2010 [11] tackled several problems in how SLDs get diagnosed by shifting some responsibilities from overwhelmed healthcare systems to schools and teachers. This law, along with guidelines for managing students with SLDs [12], prioritizes early support, encouraging the usage of technology to strengthen this process. Technology, and in particular serious games, offer a valuable resource for providing the right and timely support to both students and teachers, mainly delivering two benefits: engagement and customization. In fact, technology can offer ways to make the boring, repetitive exercises of handwriting training more engaging, avoiding feelings of frustrations which can lead to reduced participation and adherence. [13]. Secondly, technological tools can be automatically and easily personalized, offering truly targeted programs without additionally burdening teachers, who already struggle due to schools' lack of resources. In fact, especially in large classes, it can be difficult for teachers to manage all the children individually and account for everyone's difficulties; a digital solution can be the key to addressing these issues.

1.1. Serious Games

Serious games [14] are video games in which the focus is not only on entertainment, but also on achieving an objective, depending on the domain in which the game is used [15]. Generally, serious games are employed in different spheres: the top domain, as reported in [16], is education (with greater emphasis on the university [17]), followed by health and wellness.

In a school context, these games integrate elements or tasks with an educational purpose into the gameplay: the game environment is specifically designed to allow players to learn and improve their skills [18, 19]. Serious games have the potential to make learning more enjoyable, easier, interesting, effective, and efficient if well-designed [17, 19], making students not only passive recipients, but also active participants in the learning process [20, 21]. Furthermore, early use of this tool offers students different benefits: digital literacy, increased self-esteem, and a learning living experience [19]. Serious games can facilitate collaboration among schools, homes, and healthcare providers. By gathering performance data, these digital tools allow teachers, parents, and healthcare providers to monitor progress over time and, if necessary, adjust interventions.

However, the full potential of this personalization depends on real-time adaptations. Unlike traditional exercises, serious games can be designed to provide real-time feedback and eventually adapt difficulty based on performance. On the other hand, in most current applications, adaptive features are typically not automatic, relying on teachers oversight, predefined settings, or remaining not implemented. For example, Xu et al. [22] provide an overview of different serious games employed in medical education, that rely on a predefined and fixed difficulty. Moreover, Arias-Calderón et al. [23] present a serious game for nursing students that uses a progression system rather than an adaptive one.

Aware of all the aforementioned advantages, serious games are effective tools for increasing player adherence

and engagement [24]. However, sustaining engagement over time while maximizing performance is a well-known challenge in the video game world. In an educational context, this trade-off is even more relevant, as a constant practice is required.

In recent years, several strategies have been proposed. Among them, adaptive approaches have played an important role, enabling the game to adapt to the player's behavior, characteristics, and skills. Sampayo-Vargas et al. [25] highlights the effectiveness of adaptive difficulty adjustment, which increases learning outcomes. Within this context, reinforcement learning has been widely adopted because it models the interaction between the player and the environment, accounting for all possible player actions.

1.2. Reinforcement Learning

Reinforcement learning (RL) is an important online machine learning method in which two main components of the system continuously interact: the **agent** and the **environment** [26]. In the basic model, the environment, at each time t , provides to the agent a signal that represents the current state s , and then, the agent selects an action a that consequently affects and changes the environment, which leads to a new state s' . Hence, the system obtains a reward and punishment signal r from the environment. It is important to note that the agent does not know, *a priori*, which action is optimal, but the environment provides information about the quality of each action. In fact, the agent learns through interaction with the environment and by receiving rewards. The objective of a RL algorithm is to select a strategy, a policy π , that allows for obtaining the largest expected cumulative reward over time.

Interactions between environments and agents may not have explicit state transitions, so they can be modeled more simply. In this case, this kind of sequential decision-making problem can be formulated as a Multi-Armed Bandit (MAB) [27]. MAB is a variant of RL in which an agent selects an action (arm) from a set of possible ones and receives a reward, drawn from stationary or non-stationary distribution, from the environment, without considering the possible consequences on future states. Different algorithms exist to address this problem, differing in their policy.

In both problems, there is a trade-off between exploration and exploitation.

- Reinforcement Learning: the balance between these two aspects is fundamental for this kind of problem. The agent must explore better strategies that consider the impact of its actions on the future state of the environment.
- Multi-Armed Bandit: since the agent's actions do not affect the environment's future state, the balance is simplified. The agent focuses on selecting an action, and this selection is based only on past outcomes.

Both RL and its simplified version, MAB algorithms, are used in a game context, not only to maximize user performance but also to sustain long-term engagement. The next section considers related work that has used this method in similar contexts.

1.3. Related Works

Various studies have proposed the use of artificial intelligence and machine learning algorithms in educational settings in different ways. In a recent review of the literature [28], the authors identified only 15 studies that focused primarily on the use of RL techniques in an educational context. This highlights how the potential to use these algorithms in education has not yet been fully explored. From that set of studies, they identify different uses of these techniques in domains such as assistance, intelligent tutoring system, games and serious games, (math problem) automatic solver, and language learning system. Moreover, the researchers point out how typical rewards and punishments of RL approaches have to be reformulated in an educational manner, since RL algorithms tend to maximize only what it considers the most promising outcome. Different formulation of rewards, focusing not on correctness but on learning process, on improvement compared to previous attempts or on having tried different strategies, is the key to achieve a constructivist approach, where making errors is part of the learning process. This is in contrast with the behaviorist approach of RL algorithms, in which agents change their behavior to maximize a reward, selecting actions that avoid low payoff. This bypasses the value of learning through error.

Bellotti et al. [29] propose a design methodology for sandbox serious games, in which the player has to explore a virtual world to gather all the information distributed around it. Their system is able to decoupling content from their delivery strategy through an "Experience Engine", which has two possible variants, depending on the desired outcome. The first is based on genetic computation that optimizes the optimal sequence of educational objectives defined by the game author. The second used an RL agent able to support the user to achieve the

best performance by selecting the best task one at a time. The RL approach is used to sustain engagement by proposing tasks that are not too easy or too difficult, but there is no explicit measure of it.

In general, as already mentioned, few studies explored the use of an artificial intelligence system in education. Focusing on general entertainment games, other studies have investigated the topic of engagement from different perspectives. In order to address this problem, various approaches based on machine learning techniques, in particular RL and MAB methods, have been used in the literature. By gathering data from the user and analyzing them, these systems allow to dynamically adjust the difficulty of a task, maximizing the outcomes and the user participation. In the gaming domain, designing a game that adapts to the player, and not the other way around, can significantly enhance player engagement and improve performance.

As reported by Climent et al. [30], the authors propose a framework for designing a RL agent able to dynamically adjust the difficulty of a single player game (Dynamic Difficulty Adjustment - DDA). It is able to modify online the difficulty of the level increasing or decreasing, for instance, the number of enemy or the number of attacks available. They used a ϵ -greedy strategy with three different configurations: the trade-off between exploration and exploitation depends only on probability ϵ . Since their approach is developed for a single-player game, the player has to play against the RL agent. Therefore, the states of the agent reflect not only possible victory or defeat conditions but also intermediate situations, if they exist. To estimate these states, the "life" of both the player and the agent was used. For the definition of the reward, the key parameter that is taken into account to adapt difficulty is the duration of a single round: very short rounds can be symptoms of the player boredom or frustration. In particular, a positive reward was used in case the agent achieves its goal, otherwise a negative one was used, as punishment.

Another case may be the framework developed by Xue et al. [31]. The authors highlight the fact that the diversity among players could be vast: they could have different skills, experiences, gameplay style, and, more in general, preferences. Moreover, the ability of the same player could evolve in the meantime. With these premises, a video game should be able to adapt to the specific user's characteristics. They implement a DDA framework able to predict future state taking into account the current state, i.e., the level of difficulty. Their objective is also to maximize the player's engagement for the entire game duration; it is not only an adaptation problem to player skills. For this purpose, they have developed a player progression model for a level-based game in which they have two main dimensions: the player that advances to a higher level or the player who repeats the current level. They have considered as reward the expected total number of rounds a player has played during the entire game. With this reward definition, they have considered as an engagement measure the total number of rounds played: the higher this value is, higher the engagement will be.

Hence, in both [30, 31], authors used an indirect component of the reward function to obtain a kind of engagement measure.

Zhan [32] explored two potential uses of MAB algorithms: player modeling and game design optimization. In the first case, MAB algorithm can be used to model characteristic as skill level or player style, so oriented to adapt the player behavior and how this change over the game time. Also in this work, it was made explicit how machine learning techniques can be used not only to enhance the game experience, but also to keep challenging the game, avoiding frustration between players. Secondly, these algorithms can be used to optimize elements such as difficulty, level reward, or also the user interface. This is more oriented towards the personalization aspects, but the level reward optimization could also be an interesting approach. Players may be more engaged by changing the obtainable level rewards: subjects that encounter more struggles may obtain small but more frequent payoff.

Vinogradov and Harrison [33] focus on an Experience Manager (ExpM), used to offer a personalized experience to player. Gathering data of the player while playing, ExpM is able to model them and to change environment in the meantime. The main problem in this case is the fact that the environment can be biased by the ExpM self. For instance, as reported in the article, ExpM sends many monsters because sees the player killing them, but maybe it is the only way to go on in the game. So, this Experience Manager choice really does not reflect player preferences. The authors propose to model ExpM profiling as MAB, in particular using different algorithms (ϵ -greedy, UCB, and Thompson Sampling). Their objective is to have better game management to have a more engaged environment. Due to the biased environment, to recalculate player model, they have introduced what they called "*distraction actions*" (interactable object of different typology) that have the only purpose to collect data about player. These actions are used by the model as arm: if the player interacts with them receives 1 as the reward, otherwise the reward will be 0. In this way, the online method can recalculate player profile and understand what is the best arm - the player preferences - to choose. Although this solution effectively enhances personalization and player engagement, it is highly oriented toward aligning player preference rather than maximizing player performance.

Amiri and Sekhavat [34] underline how player modeling requires game data. So, they propose MAB algorithm,

an ϵ -greedy algorithm, able to determine game parameters fitting player's preferences, since it can be quite difficult to dynamically adjust those parameters at runtime, when there is not much data. They developed a sort of 3D platform, and they select environment color and movement speed to create four different themes that are the arm of the bandit. The aim of their work is to select the optimal theme for the specific player. In particular, they used the game score for a specific theme as rewards. The authors have supposed a relation between theme and score: if a player prefers a particular theme, their total point will be higher and the player will be more engaged. In this case, the authors focused more on using a MAB to select the level's aesthetic. Consequently, player engagement is treated as a product of theme preference rather than a variable that needs to be actively managed.

Preserving engagement is widely acknowledged as a challenge, not only in games field. Hence, sustaining engagement is something that affects other domains, such as clinical one. For instance, the work presented by Killian et al. [35] is focused on a digital program that supports patients with type 2 diabetes through a mobile app. The system involves sending in-app reminders, virtual coaching activities, and various requests for healthy behavior and user health updates. This article demonstrates how a MAB framework was essentially implemented to generate resource allocation strategies, the app-based intervention, to achieve the desired outcomes: optimal health status and high levels of engagement. In more detail, their approach has to take into account: evolution of the patient, engagement-health dynamic, limited observability of the data, and limited availability of resources. Therefore, their approach is a new solution: "equitable Restless MAB" (ERMAB). In MAB approaches, strategy tends to select the more promising arm, at the expense of others. In a clinical context, this approach can generate inequalities, that are already present in healthcare. With this new MAB, the method distributes resources in a more equal way. In their setting, patients are used as arms (which evolve over time also when they are not selected because the restless model) of the bandit, and a combination of engagement and clinical status value is used as reward. The MAB strategy selects the patient that will receive an intervention and then observes a reward based on available clinical data and an engagement value that depends on how much the patient interacts with the mobile app. A similar approach was used by Lee et al. [36]. The authors developed a framework capable of selecting patients for screening for hepatocellular carcinoma. Also in this case, a Restless Multi Armed Bandit was considered. Their objective was to maximize early screening considering that patient data as noisy and screening resources as limited. So, while their model are used to resource allocation where the arms are the patients and their reward is combined of engagement and health status, in this thesis MAB should focus on single user, where engagement is a variable to ensure that a game challenge remains sustainable.

1.4. Objective

In the previous section, different works in the literature were presented. In both the game world and other domains, various machine learning techniques, including RL and MAB methods, were implemented to maintain high participation levels. Open challenges still remain.

In the literary review there is a significant gap in management both performance and engagement. In almost all the papers, engagement is not considered directly in the RL algorithms but is generally related with metrics such as higher level duration or higher game scores. Hence, engagement is treated as an indirect byproduct of reward function. In other cases, RL learners are more oriented toward personalization than toward maximizing players' performance: these solutions actually lead to a more immersive environment but do not directly address player skills, even though engagement and ability improvement are related. Even in healthcare domain, where ERMAB solutions are employed for resource allocations, engagement remains a component of reward than a stand-alone variable. Moreover, these techniques are rarely used in serious games, where the control of engagement, in an explicit way, is fundamental to achieving results over time, particularly if the game is aimed at education. Another main challenge related to educational games persists: each student is unique, with their own characteristics and learning difficulties, which requires a system capable of automatically adapting to their individual needs for practice, with direct control over the level of participation.

Given these premises, this work aims to develop an RL-based system to automatically provide difficulty progression in a game-based handwriting training, trained and evaluated by data coming from a game-based handwriting training intervention in primary school children. To achieve this primary objective, the following goals are pursued sequentially:

1. identifying the MAB algorithm that best suits the static problem formulation, evaluating different approaches under controlled simulation conditions;
2. analyzing different scenarios and data aggregation strategies to identify the best configuration in which to train the selected MAB system;
3. extending the analysis to a dynamic environment, investigating how the system adapts to non-stationary reward distributions reflecting the evolving skill level of the child;

4. designing the deployment of the system in a real-world educational context and outlining a protocol for a future longitudinal study to validate its effectiveness in clinical and school settings.

2. Materials and Methods

2.1. Apparatus

2.1.1 Number Tracing

Number Tracing is a handwriting video game developed in Unity and deployed on a 10.6-inch Samsung S6 Lite tablet, equipped with the stylus pen, the S-Pen. The game, developed before this study, was co-designed with neuropsychiatrists to help children develop fine motor control and enhance precision needed for handwriting. It is set on a beach, where a snake guides players to trace numbers.

In Number Tracing, a number appears on the screen, and the player has to select the corresponding quantity of fish from a set. Then, the player has to trace the number, following the path shown from a snake.

Difficulty is selected manually, and evolves as following:

- **Easy:** numbers' edges maintain a standard width;
- **Medium:** the path width is unchanged, before tracing children encounter two versions of the same number: the correct one and a mirrored version.
- **Difficult:** in this case, the width is reduced in respect to the previous difficulties, requiring higher precision.

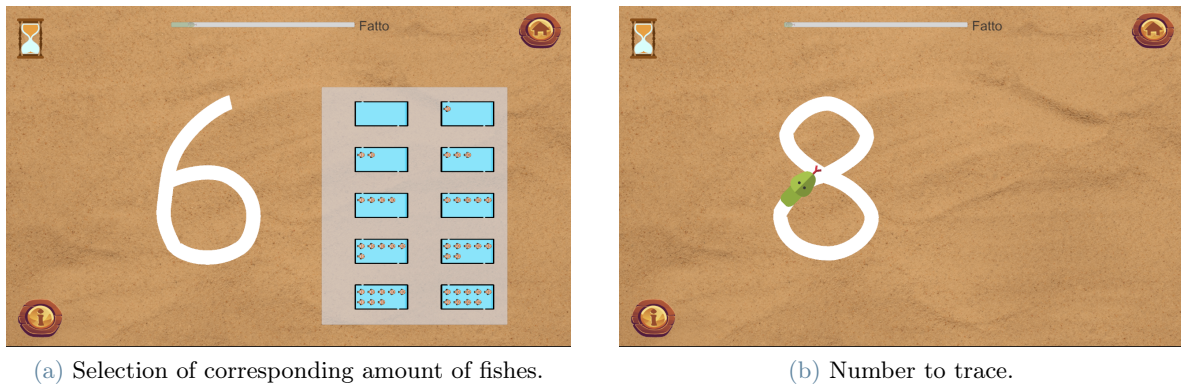


Figure 1: Number Tracing gameplay.

2.2. Participants and Protocol

The study involved first-grade children recruited from four classes across two primary schools in the Varese province. The trial included a three-month intervention study in which each participant had to take part in a 20-minute training session twice a week.

The handwriting games, including Number Tracing, were administered following a predetermined protocol from February 17th, 2025, to May 5th, 2025. Researchers and teachers were present in the classroom for each session, but they did not influence the training besides offering explanations before new tasks and clarifications when needed. Training sessions lasted 20 to 35 minutes.

The protocol was approved by Politecnico di Milano's Ethical Committee n. 52/2024.

2.3. Game Features Extraction and Selection

The Samsung S-Pen was used to collect data related to the measurements of tilt, azimuth, and pressure, and the tablet was used to collect the input data of the pen, including the coordinates in the screen space. From a previous work, feature extraction follows a three-stage process.

1. **Data Processing:** all invalid coordinates and pressure values under a threshold (below 0.05-0.09) were removed. In addition, all sessions with lower data (under 10 valid) were excluded. Then, the remaining valid data are temporally resampled with a linear interpolation to obtain 30 Hz sampling across all pen parameters. In particular, this was necessary to avoid temporal irregularity between the Unity’s refresh rate, around 30 frames per second, and S Pen sampling rate, which is higher.
2. **Pen-Based Features Extraction:** operates on the kinematics data to obtain different movement characteristic that are in common with all the handwriting games. They are reported in Table 1.
3. **Game Outcome Feature Investigation:** extracts performance metrics that are unique for each game. Referring particularly to Number Tracing:
 - (a) Accuracy (%): percentage of pen trace within the edges of the path;
 - (b) Strokes ratio (dimensionless): compares the executed strokes with expected ones;

Once all the features have been extracted, a further **data cleaning** was performed, removing data outside the expected ranges. Then, two features were selected to model the problem.

Table 1: Summary of extracted features

Feature	Description	Units
<i>Basic Kinematic Features</i>		
Median tilt	Median pen tilt angle relative to tablet surface	degrees
IQR tilt	Interquartile range of pen tilt angles	degrees
Max pressure	Maximum pressure applied during session	normalized
Median pressure	Median pressure applied during session	normalized
IQR pressure	Interquartile range of pressure values	normalized
Median speed	Median pen movement velocity	pixels/s
IQR speed	Interquartile range of movement speeds	pixels/s
Max speed	95th percentile of movement speeds	pixels/s
Median acceleration	Median change in velocity	pixels/s ²
Centroids	Mean x,y coordinates of pen trajectory	pixels
<i>Temporal Behavior Features</i>		
% pauses on tablet	Time spent in lowest 5% of speed range	%
% time in air	Time with pen lifted (pressure < threshold)	%
Time	Total session duration	seconds
# tracts	Number of continuous pen-on-tablet segments	count
<i>Movement Quality Features</i>		
Log dimensionless jerk	Jerk-based measure of movement smoothness	none
Average SNVPD	Signal-to-Noise velocity peaks difference	count
<i>Frequency Domain Features</i>		
Speed dominant frequency	Primary frequency component of speed signal from FFT analysis	Hz
Speed bandwidth	Frequency range at half-power of dominant peak	Hz
Pressure dominant frequency	Primary frequency component of pressure signal from FFT	Hz
Pressure bandwidth	Frequency range at half-power of pressure dominant peak	Hz
Tilt dominant frequency	Primary frequency component of tilt signal from FFT	Hz
Tilt bandwidth	Frequency range at half-power of tilt dominant peak	Hz

2.4. Static Modeling of the Problem

The goal of this work is to maximize performance over time, sustaining a high level of player engagement. There is a need for an agent that, in each step, selects the best Number Tracing level that maximizes a performance feature of the serious game over time, respecting a constraint over the level of engagement. For these reasons, the Number Tracing serious game can be modeled as a constrained MAB problem, similar to the version developed in [37] and [38]. To define the model, game elements were mapped onto the MAB framework.

- **Actions (Arms):** the agent action set consists in the three difficulty levels of Number Tracing game, with each arm representing the level of difficulty (easy, medium, and hard).

- **Reward:** is defined as a performance metrics. It should be a feature that reflects how effectively the player completes the task of the given level.
- **Constraint:** to maintain the player motivated, an engagement measure, chosen from game features, was defined as constraint. Therefore, before selecting an arm that maximizes the reward, the agent must ensure that it respects the imposed constraint. This implies that it must first prioritize engagement and then choose an action among those satisfying the constraint.
- **Policy:** is the action-selection strategy employed by the agent. Each algorithm has its own rule for determining which arm to pull. In order to evaluate different strategies, different MAB algorithms were tested: ϵ -greedy (EG), Upper Confidence Bound (UCB), and Thompson Sampling (TS).
- **Players:** were modeled as static players. This is a strong simplification because, in reality, players enhance their ability by continuing to play. However, it was possible to test algorithms in a static environment to analyze performances under controlled and fixed data distributions.

2.4.1 Multi-Armed Bandit

Multi-Armed Bandit is a sequential decision-making problem in which there is a *learner* and an *environment*, as already explained in Section 1.2 [39]. A typical MAB algorithm, the learner has to choose - *pull*, in each round t , a possible action $a_t \in \mathcal{A}$, i.e., a bandit *arm*. The selection process is repeated for a total of T rounds, where T is the time horizon. In each step t , the algorithm receives a random reward $r_t \in \mathbb{R}$ from the environment, independently drawn from an unknown and stationary probability distribution associated with the selected arm. In these terms, rewards are assumed to be independent and identically distributed (IID rewards) for each arm [27]. The objective of a MAB algorithm is to maximize the expected cumulative reward, over the finite time horizon T :

$$\max_{\pi} \mathbb{E} \left[\sum_{t=1}^T r_t \right], \quad (1)$$

where π is the policy used, i.e., the strategy used by the learner to select an arm. Notice that maximizing Equation (1) is equivalent to minimizing cumulative regret. Indeed, in this work, the cumulative pseudo-regret was taken into account:

$$R_T = T \cdot \mu^* - \mathbb{E} \left[\sum_{t=1}^T r_t \right], \quad \text{where } \mu^* = \max_{a \in \mathcal{A}} \mu_a. \quad (2)$$

In practice, the cumulative pseudo-regret is the difference, at each step, between the maximum expected reward of the best arm and the expected reward of the selected arm. Therefore, regret is a measure of how much the learner loses if they select an arm that is not the optimal one. Hence, this type of algorithm needs to balance *exploration* of arms that it does not yet know and *exploitation* of arms that it already knows, which will give it the maximum reward [27].

2.4.2 General Framework

From the game, two features were identified as measures of performance and engagement.

- For the performance, *round accuracy* of the trace was chosen. Accuracy refers to how well a specific level was executed and should be maximized, especially in a serious game. Therefore, the round accuracy can be used as a reward.
- *Time-in-air* feature was selected as engagement measurement $e_t = T_{in\ air_t}$. It can be reasonably assumed that a longer time player who passes with a pencil in the air rather than on the tablet will be less engaged due to boredom, frustration, and distraction. In this context, lower time-in-air values are preferable.

As a constrained MAB, a θ threshold was adopted and expresses a time-in-air amount that should not be overcome in order to sustain involvement. The θ threshold was set at the 75th percentile of the data distribution related to the time-in-air feature. This value was determined through a descriptive statistical analysis and it was used to evaluate how many times each level can be considered unfeasible in different data aggregation setups.

The three algorithms have in common the same framework, shown in Algorithm 1.

For the first three steps, an exploration phase begins, since the agent has no information. So, it plays each arm at least one time ($|\mathcal{A}| = 3$) and receives both accuracy/reward and time-in-air (engagement) value. Then, after the first exploration phase, in each round t , the learner evaluates if a certain time-in-air quantity $eng_{i,t-1} \leq \theta$, where eng depends on the algorithm. Inequality leaves out arms with a time-in-air above θ and a low engagement. In this way, it defines $\hat{\mathcal{A}}_{feas} \subseteq \mathcal{A}$, a subset of $\mathcal{A}$ that considers only feasible arms. From this new set, the learner selects an arm a_i , choosing depending on its policy based on $acc_{i,t-1}$. Finally, it receives, for the selected arm, a stochastic reward and an engagement value, which update the $acc_{i,t}$ and $eng_{i,t}$ amounts that will be used in the

Algorithm 1 Framework

Require: Input: $\mathcal{A}, \theta, T$

```
1: Initialize  $acc_{i,t=0}, eng_{i,t=0} \leftarrow 0$ 
2: for each  $a_i \in \mathcal{A}$  do
3:    $a_t \leftarrow a_i$ 
4:   Observe  $r_{a_t,t}, e_{a_t,t}$ 
5:   Update statistics of arm  $a_t$ 
6:    $t \leftarrow t + 1$ 
7: end for
8: for  $t = |\mathcal{A}| + 1$  to  $T$  do
9:    $\tilde{\mathcal{A}}_{feas} \leftarrow \{a_i \in \mathcal{A} \mid eng_{i,t-1} \leq \theta\}$ 
10:  if  $\tilde{\mathcal{A}}_{feas} = \emptyset$  then
11:    Choose random  $a_t \in \mathcal{A}$ 
12:  else
13:    Choose  $a_t \in \tilde{\mathcal{A}}_{feas}$  according to  $\pi$  based on  $acc_{i,t-1}$ 
14:  end if
15:  Observe  $r_{a_t,t}, e_{a_t,t}$ 
16:  Update statistics of arm  $a_t$ 
17: end for
```

evaluation in the next round t . The update of these quantities is handled differently by the algorithms. If no arm is feasible, i.e. $\tilde{\mathcal{A}}_{feas} = \emptyset$, the learner will select a random arm from the set $\mathcal{A}$, to allow the decision-maker to explore.

Moreover, to evaluate performance using the real average of the data distribution, a subset $\mathcal{A}_{feas} \subseteq \mathcal{A}$ was defined as:

$$\mathcal{A}_{feas} = \{a_i \in \mathcal{A} \mid \mu_e(a_i) \leq \theta\}. \quad (3)$$

Note that: $\tilde{\mathcal{A}}_{feas} \neq \mathcal{A}_{feas}$, the first one is what is calculated by algorithm in each step, the second is calculated with a priori knowledge: they may not coincide. If all arm overcome the θ threshold, the Equation (3) becomes:

$$\mathcal{A}_{feas} = \arg \min_{a \in \mathcal{A}} \mu_e(a). \quad (4)$$

In this case, the arm with the least amount of time with the pen lifted will be considered the optimum one, prioritizing maximizing the time spent in the game rather than performance.

From the $\mathcal{A}_{feas}$ subset, the best arm was calculated as:

$$\mu^* = \max_{a \in \mathcal{A}_{feas}} \mu_a(a). \quad (5)$$

In Equations (3), (4), and (5), real means were used but it is important to highlight that $\mathcal{A}_{feas}$ and the true average of the data are unknown to the learner. These are used only to evaluate the performance of each algorithm.

Having defined this new set $\mathcal{A}_{feas}$, if the agent selects an arm $a \notin \mathcal{A}_{feas}$, in the regret calculation, a penalty of 0 accuracy was applied if the learner selects an arm that overcomes the θ threshold. Equation (2) becomes:

$$\begin{cases} R_T = T \cdot \mu^* - \mathbb{E} \left[\sum_{t=1}^T r_t \right] & \text{when } a_t \in \mathcal{A}_{feas} \\ R_T = T \cdot \mu^* - 0 & \text{when } a_t \notin \mathcal{A}_{feas} \end{cases}. \quad (6)$$

2.4.3 Selection of the Best Algorithm

As anticipated, three different algorithms were taken into account in order to test different action-selection strategies.

Epsilon-greedy ϵ -greedy is one of the most common and simple MAB algorithms. Its peculiarity lies in the presence of the probability parameter ϵ . The trade-off between exploration and exploitation depends on this probability and, in fact:

- with probability $1 - \epsilon$: the agent always chooses the best arm, i.e. the arm with the highest estimated mean of the reward at that time step, from the subset $\tilde{\mathcal{A}}_{feas}$ (exploitation);
- with probability ϵ : the agent selects a random arm from the set $\mathcal{A}$ (exploration).

This mechanism guaranties that the decision-maker selects the best arm most of the time, with an expected reward that converges to the true mean reward of the best arm from the very first steps. Despite that, in each round t , there is a ϵ probability that the policy selects a sub-optimal arm. The cumulative regret linearly increases: the exploration phase continues over the time horizon T [39].

Algorithm 2 Constrained ϵ -greedy

Require: Input: $\mathcal{A}, \epsilon, \theta, T$

- 1: Initialize $acc_{i,t=0}, eng_{i,t=0} \leftarrow 0$
- 2: **for** each $a_i \in \mathcal{A}$ **do**
- 3: $a_t \leftarrow a_i$
- 4: Observe $r_{a_t,t}, e_{a_t,t}$
- 5: Update statistics of arm a_t
- 6: $t \leftarrow t + 1$
- 7: **end for**
- 8: **for** $t = |\mathcal{A}| + 1$ **to** T **do**
- 9: $\tilde{\mathcal{A}}_{feas} \leftarrow \{a_i \in \mathcal{A} \mid \hat{e}_{a,t-1} \leq \theta\}$
- 10: **if** $\tilde{\mathcal{A}}_{feas} = \emptyset$ **then**
- 11: Choose $a_t \in \mathcal{A}$
- 12: **else**
- 13: Draw $rand \sim U(0, 1)$
- 14: **if** $rand > \epsilon$ **then**
- 15: $a_t \leftarrow \arg \max_{a \in \tilde{\mathcal{A}}_{feas}} \hat{r}_{a,t-1}$
- 16: **else**
- 17: Choose $a_t \in \mathcal{A}$
- 18: **end if**
- 19: **end if**
- 20: Observe $r_{a_t,t}, e_{a_t,t}$
- 21: Update $\hat{r}_{a,t}, \hat{e}_{a,t}$
- 22: **end for**

In this case, as shown in Algorithm 2, the estimated means of an arm in the time step $t - 1$ were used both for reward and for time-in-air. Hence, the estimated mean of time-in-air $\hat{e}_{a,t-1}$ was used as a comparison with the threshold, and the estimated mean of accuracy $\hat{r}_{a,t-1}$ was employed in the selection of the maximum.

Upper Confidence Bound The principle on which UCB is based is optimism over uncertainty [39]. Indeed, when data is not enough, this approach encourages exploration. After trying and collecting information from the arms, statistics can be updated, and the learner can make more conscious decisions. The optimism principle is based on using data from the policy observed, adding the *upper confidence bound*, an exploration bonus: an overestimation of the observed means of the arms, formally:

$$\hat{\mu}_{t-1} + \sqrt{\frac{2\ln(t)}{N_a}} = \hat{\mu}_{t-1} + \Delta i. \quad (7)$$

As shown in Equation (7), the confidence bound depends on the round t and on the number of times N an arm was chosen by the learner [39]. Making a concrete example: a not optimal arm can be played only if its confidence bound is larger than the optimal one. This suboptimal arm can not be played for so long because the more it is played, the more information is acquired by the learner, and the more the confidence bound decreases. Moreover, the term $\ln(t)$ guarantees a logarithmic growth:

- when t is small, growth occurs rapidly, favoring exploration;
- when t is very large, growth slows, favoring exploitation.

Specifically because of this behavior, UCB tends to have a regret that increases rapidly in the first few steps t and then stabilizes over the time horizon T . Furthermore, since it is deterministic in its choices because they

depend only on the estimated mean and the confidence bound, it has little variability over multiple repeated runs.

In the constrained version, this optimistic principle was used for both accuracy and time-in-air.

$$\hat{\mu}_{accuracy_{t-1}} + \sqrt{\frac{2\ln(t)}{N_a}}; \quad \hat{\mu}_{time\ in\ air_{t-1}} - \sqrt{\frac{2\ln(t)}{N_a}}. \quad (8)$$

For the engagement, a Lower Confidence Bound (LCB) was used, so it is an underestimation of the observed mean of time-in-air. Contrary to what it may seem, the use of an LCB is optimistic about engagement: in each step, learner identifies $\tilde{\mathcal{A}}_{feas}$ excluding arm whose LCB is above the threshold, in other words, it excludes arm whose worst case (for the time-in-air) - the underestimation - is over the threshold. From an engagement point of view, this approach is equivalent to saying that the learner excludes arms with an overestimation of the engagement. In this sense, it can be said that the framework is not very stringent on the constraint, as it is accepted to exclude an arm only when its worst value exceeds the threshold. This line of action differs slightly from the constrained version of [37], which implemented an Optimistic Pessimistic Bandit. The authors are pessimistic about the cost function (the equivalent of engagement); in fact, they used an upper confidence bound for both the cost and the reward. They are much more conservative in their approach to the constraint, prioritizing safety over risk within the present framework.

As a result, in the UCB, whose pseudo-code is provided in Algorithm 3, the accuracy and engagement amounts from the general framework are, respectively, overestimated and underestimated. After the learner receives both reward and engagement value from the environments, both confidence bounds are updated for each arm.

Algorithm 3 Constrained UCB

Require: Input: $\mathcal{A}, \theta, T$

```

1: Initialize  $acc_{i,t=0}, eng_{i,t=0}, N_i \leftarrow 0$ 
2: for each  $a_i \in \mathcal{A}$  do
3:    $a_t \leftarrow a_i$ 
4:   Observe  $r_{a_t,t}, e_{a_t,t}$ 
5:   Update statistics of arm  $a_t$ 
6:    $t \leftarrow t + 1$ 
7: end for
8: for  $t = |\mathcal{A}| + 1$  to  $T$  do
9:    $\tilde{\mathcal{A}}_{feas} \leftarrow \left\{ a_i \in \mathcal{A} \mid \hat{e}_{i,t-1} - \sqrt{\frac{2\ln(t)}{N_i}} \leq \theta \right\}$ 
10:  if  $\tilde{\mathcal{A}}_{feas} = \emptyset$  then
11:    Choose  $a_t \in \mathcal{A}$ 
12:  else
13:     $a_t \leftarrow \arg \max_{a \in \tilde{\mathcal{A}}_{feas}} \hat{r}_{i,t-1} + \sqrt{\frac{2\ln(t)}{N_i}}$ 
14:  end if
15:  Observe  $r_{a_t,t}, e_{a_t,t}$ 
16:  Update both Confidence Bound
17: end for
```

Thompson Sampling At the base of Thompson Sampling [40] there is the beta distribution $\mathcal{B}$, a continuous probability distribution defined on the interval $[0, 1]$. Its density function, given $x \in [0, 1]$, is defined as:

$$f(x) = x^\alpha (1-x)^\beta \cdot \frac{1}{\mathcal{B}(\alpha+1, \beta+1)} \quad \text{with } \alpha, \beta > 0. \quad (9)$$

In Thompson Sampling, each action is associated with a beta distribution, which acts as probability model of the expected reward. In particular, this distribution depends on the number of successes α and failures β for each arm. Figure 2 shows an example of the beta distribution in which α and β parameters are respectively 4 and 8.

The learner draws samples from every arm distribution and chooses the arm with the maximum reward probability. Then, for the only selected arm a , the decision-maker updates both α and β parameters, so the so-called

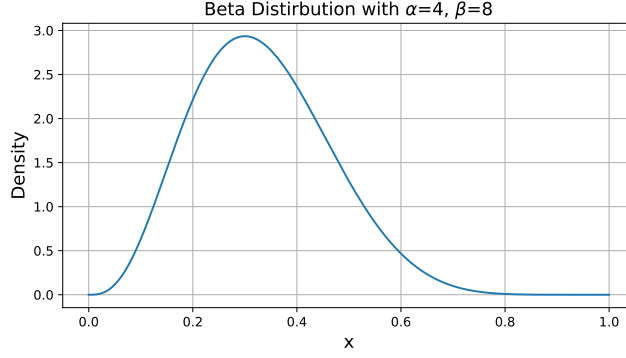


Figure 2: Example of beta distribution.

beta posterior is updated according to Bayes' rule. In particular:

$$(\alpha_k, \beta_k) \leftarrow \begin{cases} (\alpha_k, \beta_k) & \text{if } a_t \neq a \\ (\alpha_k, \beta_k) + (r_t, 1 - r_t) & \text{if } a_t = a \end{cases}. \quad (10)$$

Note that the expected value and variance of the beta function are:

$$\mathbb{E}[p] = \frac{\alpha}{\alpha + \beta}; \quad \text{Var}(p) = \frac{\alpha\beta}{(\alpha + \beta)^2(\alpha + \beta + 1)}. \quad (11)$$

Overall, as α and β increase, the distribution becomes more concentrated around the expected value and the uncertainty decreases. In the first step, the prior distribution is $\mathcal{B}(1, 1)$ - the uniform case $\mathcal{U}(0, 1)$ - in which every value has the same probability of being drawn. Then, when the selected arm receives the reward, the distribution can be updated, and therefore, in the next step, the learner draws probability from the posterior.

For the framework, the object of this work, the beta distribution was used to model both reward and engagement. In this case, *acc* and *eng* amounts are the samples that the learner draws from. The huge difference from a standard Thompson Sampling algorithm is the method used to update the posterior. Usually, in bandit problems, Bernoulli arms are used: they give the learner rewards that can be only 0 or 1. Accuracy and time-in-air are continuous values ranging from 0 to 1. Consequently, considering observed reward and time-in-air values as probabilities of obtaining success and drawing $p \sim \mathcal{U}(0, 1)$:

- if $p >$ observed reward/time-in-air: learner updates β , because we have obtained a failure;
- if $p <$ observed reward/time-in-air: vice versa, learner updates α .

The Constrained TS pseudo-code is presented in Algorithm 4.

A constrained Thompson Sampling was developed in [41]. This version was not considered because the work was recently published and their approach does not fit the needs of this thesis.

2.4.4 Metrics Performance

For the evaluation of the algorithms, different performance metrics have been taken into account.

- **Cumulative reward:** maximizing this quantity is the objective of the MAB problem. It is, as already written, the sum over time of the reward observed by the learner, once it selects an arm. It is a direct measure of an algorithm's effectiveness in maximizing rewards.
- **Mean reward:** measures the average reward that the algorithm collected over the time horizon T . It is an important evaluation metric: a higher mean reward means that the agent is able to select arms that return high payoff. Let to understand how much and how quickly an algorithm converges to an optimal value μ^* , defined in Equation (5).
- **Cumulative regret:** indicates the performance gap between what the agent selects and the optimal policy. It quantifies the loss that the agent has when selects a suboptimal action during the online learning process. It is a sort of cost measure: it reflects how much the algorithm is efficient to balance the trade-off between exploration and exploitation. For this reason, the main objective of a MAB problem is to minimize cumulative regret.

Algorithm 4 Constrained Thompson Sampling

Require: Input: $\mathcal{A}, \theta, T$

```
1: Initialize  $acc_{i,t=0}, eng_{i,t=0} \leftarrow 0$ 
2:  $\alpha_i^{acc}, \beta_i^{acc} \leftarrow 1$ 
3:  $\alpha_i^{eng}, \beta_i^{eng} \leftarrow 1$ 
4: for each  $a_i \in \mathcal{A}$  do
5:    $a_t \leftarrow a_i$ 
6:   Observe  $r_{a_t,t}, e_{a_t,t}$ 
7:   Update statistics of arm  $a_t$ 
8:    $t \leftarrow t + 1$ 
9: end for
10: for  $t = |\mathcal{A}| + 1$  to  $T$  do
11:   Draw  $sample_{acc_i} \sim \mathcal{B}(\alpha_i^{acc}, \beta_i^{acc})$ 
12:   Draw  $sample_{eng_i} \sim \mathcal{B}(\alpha_i^{eng}, \beta_i^{eng})$ 
13:    $\tilde{\mathcal{A}}_{feas} \leftarrow \{a_i \in \mathcal{A} \mid sample_{eng_i} \leq \theta\}$ 
14:   if  $\tilde{\mathcal{A}}_{feas} = \emptyset$  then
15:     Choose  $a_t \in \mathcal{A}$ 
16:   else
17:      $a_t \leftarrow \arg \max_{a \in \tilde{\mathcal{A}}_{feas}} sample_{acc_i}$ 
18:   end if
19:   Observe  $r_{a_t,t}, e_{a_t,t}$ 
20:   Update  $\alpha_{a_t}^{acc}, \beta_{a_t}^{acc}$ 
21:   Update  $\alpha_{a_t}^{eng}, \beta_{a_t}^{eng}$ 
22: end for
```

- **Arm selection frequency:** in addition to the classical evaluation criteria, it was taking into account also how frequently each arm was selected by the learner. This kind of visualization allows a better comprehension of the balance of exploration and exploitation, and of the learning dynamics. Furthermore, it shows how much the agent has selected unfeasible or sub-optimal arms.

Once these quantitative indicators of algorithmic performance were identified, multiple independent runs n were conducted. Multiple iterations are critical in this context, since algorithms are intrinsically stochastic. A single run can be influenced by multiple factors, including different decision-making and the stochastic nature of the reward mechanism; it may not be really representative because of its randomness. By running numerous trials, it is possible to better capture the range of possible outcomes and quantify the variability in the performance metrics of the algorithms. Once the results of these runs were obtained, the 95% confidence intervals were calculated as follows:

$$CI_{0.95} = \bar{X} \pm 1.96 \cdot \frac{\sigma}{\sqrt{n}}, \quad (12)$$

where $\bar{X}$ and σ are, respectively, the average and standard deviation of the cumulative reward, the mean reward, the cumulative regret, and the arm selection frequency. Equation (12) was used assuming that the metrics follow a normal distribution, an assumption justified by the central limit theorem. The confidence intervals are fundamental for assessing the performance of the algorithms: they measure the uncertainty around the average of the metrics. Narrow confidence intervals indicate a lower variability and a high stability of the means.

At this point, with these indicators and with the corresponding confidence intervals, it is possible to analyze which algorithm has the optimum behavior in terms of the average mean reward and the average cumulative regret over the time horizon T , considering the variability between a run and another.

2.4.5 Selection of the Best Configuration

Three different possible test configurations were considered to evaluate MAB methods performance.

The first configuration **individual**, in which each player was considered in their uniqueness: the data used for the simulation of the algorithm are individual. The advantage of using this configuration is assessing the algorithm when it considers children own behavior and characteristic: this allows it to offer a highly personalized experience, highlighting the difference between each player. In total, 63 simulations were performed,

corresponding to the number of children considered after filtering the original dataset. Specifically, participants who had played only one level were excluded. This setting offers the highest granularity and the most detailed view of the MAB problem.

The second configuration is **children-aggregated**. To test the algorithms, all the collected data were used to perform a single simulation without distinguishing between different children. In this way, it can be possible to evaluate the algorithm in a general scenario where individual characteristics were ignored.

Finally, a third **cluster-based** configuration was considered. A clustering was performed according to the features of Number Tracing game, also used in the modeling of MAB problem: accuracy and time-in-air. In particular, a child-level aggregated analysis was conducted: missing values were imputed with median imputation, and features were standardized (z-score), so that no data dominate. Then, K-Means and K-Medoids were tested with $2 \leq K \leq 6$, both with and without PCA dimensionality reduction (explained variance threshold $\geq 80\%$). Cluster quality was evaluated using the Silhouette score, Calinski-Harabasz index, and the percentage of significant pairwise comparisons (Mann-Whitney U test with Bonferroni correction). Finally, the optimal configuration was selected based on the highest Silhouette score: in particular, K-Medoids with $k=2$, and without PCA reduction was identified as the best one (results are reported in Table 2).

Method	k	Silhouette score	Calinski-Harabasz index	Statistical Difference
K-Medoids	2	0.313	18	83.3%

Table 2: Results of the best clustering configuration.

The two resulting clusters were as follows:

1. **Proficient children**: considers children who have a high level of accuracy and also a very short time with the pen in air;
2. **Struggling children**: corresponds to children who have greater difficulty in completing the trace and have a longer time-in-air value.

Cluster	Total children	Data amount
Proficient children	55	5361
Struggling children	11	697

Table 3: Clusters composition.

Data were divided into two different clusters (Table 3) and, as a consequence, two different simulations were performed. This setting allows to assess algorithms performance when different group of player exist, each with different behavior patterns. It is a sort of comprises between the previous configurations: individual-specific and considering the entire dataset as a unique child. It is important to note that these clusters are not known a priori by the algorithm: the resulting clusters come from a posterior and *offline* analysis. In a real *online* learning context, information about any cluster would not be available to agents. Despite that, testing different MAB methods in this way, let understand their behavior with supplementary information.

To summarize, there are three configurations in which the following amount of data is considered (Table 4):

Configurations	Filters	Total children	Data amount	Total run
Individual	Removed: invalid data + data belonging to players who have only played one level of three	63	5873	$63 \cdot n_{run}$
Aggregated	Removed: only invalid data	66	6058	$1 \cdot n_{run}$
Cluster	Removed: only invalid data	66 (55+11)	6058	$2 \cdot n_{run}$

Table 4: Summary of data available in which processing, explained in Section 2.3, were applied.

2.4.6 Evaluation Protocol

Using different test configurations allows to evaluate MAB algorithms in different situations and context, highlighting variability in outcomes when they considered individual, grouped, or global dataset.

Each algorithm is tested with a time horizon of $T = 10,000$, repeating the experiment for 10 runs. As already said, reward and engagement value are the features of the serious game: accuracy and time-in-air. In order to understand algorithms behavior, they are analyzed performing uniform random resampling from specific filtered dataset for each configuration. All simulations were performed in Python 3.10.2.

Then, once taking into account the correct metrics to evaluate algorithms performance and taking into account the configuration in which the algorithms are tested, it is possible to understand which is the best combination.

To this end, in each configuration, for each run, the value of cumulative reward and cumulative regret, corresponding to the real number of sessions played, were considered. Depending on the configuration, values for child/cluster were summed.

1. Individual configuration: in each run, the corresponding values of both performance metrics were summed for each participant, taking into consideration the real session until each student have played.
2. Cluster configuration: since there are two clusters, cumulative reward and regret were considered in time step corresponding to the total session played for each cluster. The values obtained for both groups were summed.
3. Aggregated configuration: both performance metrics are considered at time step equal to the total sessions.

Then, mean, standard deviation, 95% confidence intervals were calculated over n runs. Once the results were obtained, it was possible to compare the different algorithms in the different configurations to establish which pair was the best.

2.5. Dynamic Modeling of the Problem

Up to now, the static case was considered. A static environment does not reflect reality as players continue to play, learn, and improve. Because of this reason, since the static case is not representative of the real world, moving to a dynamic model of MAB problem can be a solution.

2.5.1 General Framework

The formulation of the modeling and the structures of the algorithm remain the same, as already discussed in 2.4.1. What it is possible to manage, in order to simulate a time advancement, is the dataset. In the static case, all the data were used without considering when they were collected. In a dynamic context, the filter dataset used in **aggregated** configuration was:

- arranged in chronological order considering all players as a single player;
- divided into *time windows* that contain at least n samples for each difficulty arm.

The main idea beyond this approach is the possibility of having data that are temporally coherent, since the subdivision stops a time t^* of the last sample in the window. In this new strategy, algorithms can obtain the reward and the time-in-air value only from the time window and not from the entire set of data. Given the nonhomogeneous distribution of the data, the initial windows were very large, as it was impossible to find samples for the easy difficulty level, with $n = 30$. For this reason, and to improve clarity in the graphs reported in the results, it was decided to add an additional data filter. In this way, the starting point is the time step $t = 3366$, that allows all windows to have more or less a fairly uniform number of samples per levels. In practice, the first two original windows were not taken into account, so the starting point became the third window in the original dataset (Figure 3).

As shown in the figure, it was impossible to obtain windows with a homogeneous distribution of data, since the hard level was initially the most popular level among players. Although this kind of filter clearly eliminates more than half of the data, for modeling a consistent time trend, this approach results the best solution in this way. In Table 5, the remaining available data are reported.

Taking these considerations into account, in the following Table 6 it is possible to view the statistical characteristics of the composition of a window. Note that, in the last window (number 17) there are no available data for easy level.

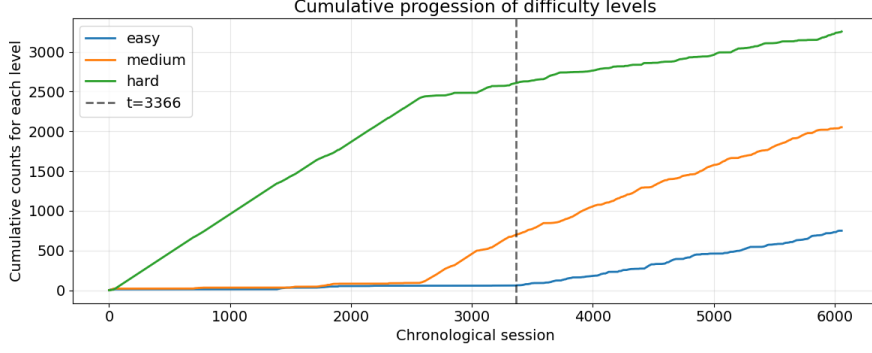


Figure 3: Cumulative counts for easy, medium and arm in chronological order of data.

Configurations	Filters	Total children	Data amount	Total run
Aggregated	Removed: only invalid data + the first two windows	66	2692	$1 \cdot n_{run}$

Table 5: Summary of data available in which pre-processing, explained in Section 2.3, were applied. Moreover, another filter was added.

Difficulty	Window count	Mean	Std	Min	Q1	Median	Q3	Max
easy	16	43.063	13.193	30	30	40	54.5	68
medium	17	79.647	41.450	3	46	82	105	168
hard	17	38.176	18.545	9	30	30	35	78

Table 6: Statistical characteristic relating to window composition for each level of difficulty.

What makes this approach dynamic is the transition from a window to the next. It takes place when the current step of an algorithm corresponds to the time step t^* . Each window has its own real mean μ_a and μ_e and, as a consequence, at each transition from one to the following, a recalculation of the set of the feasible arm $\mathcal{A}_{feas}$ (Equations (3) and (4)) and the best arm (Equation (5)) is required to correctly evaluate the performance metrics.

2.5.2 Selection of the Best Algorithm

In addition to the above-mentioned ϵ -greedy, UCB, and Thompson Sampling algorithms, in this section, Linear UCB (LinUCB) was also taken into consideration. In respect with the others, LinUCB adapts its estimated parameters to new observations. This makes it suitable for dynamic approaches: in fact, when the context is "new", uncertainty increases, allowing for exploration of new actions.

LinUCB LinUCB is a **contextual** bandit algorithm [42]. The main difference between a contextual bandit and a classical bandit problem is the fact that in this case the learner selects arms based on contextual information of the users (and arms) and based on the arm that maximizes the expected mean reward.

In a contextual bandit problem, the learner observes the data about the current user, possible actions from the set $\mathcal{A}_{\sqcup}$, and their feature vector $\mathbf{x}_{t,a}$, i.e, the context. The algorithm selects $a_t \in \mathcal{A}_{\sqcup}$ that maximizes the expected mean reward, based on the estimated mean of the previous steps. Then, the environment gives it a reward r_{t,a_t} , whose expectation is:

$$\mathbb{E}[r_{t,a}, \mathbf{x}_{t,a}] = \mathbf{x}_{t,a}^\top \theta_a^*. \quad (13)$$

Equation (13) depends on the characteristic vector and on the coefficient vector θ_a^* , which estimates the linear relationship between features and reward. Since the parameters are arm-specific, this model is called **disjoint** LinUCB. An estimation of coefficients θ_a can be obtained applying the ridge regression:

$$\hat{\theta}_a = (\mathbf{D}_a^\top \mathbf{D}_a + \mathbf{I}_d)^{-1} \mathbf{D}_a^\top \mathbf{c}_a, \quad (14)$$

where $\mathbf{D}_a$ is the matrix corresponding to the context observed in the previous steps, $\mathbf{I}$ is the identity matrix, and $\mathbf{c}_a$ is the matrix corresponding to the observed rewards. The same Equation (14) can be rewritten as:

$$\hat{\theta}_a = \mathbf{A}^{-1} \mathbf{b}_a, \quad (15)$$

where $\mathbf{A}_a$ is the first part of the equation $(\mathbf{D}_a^\top \mathbf{D}_a + \mathbf{I}_d)^{-1}$ and $\mathbf{b}_a$ is equal to $\mathbf{D}_a^\top \mathbf{c}_a$. As a classic UCB method always chooses the arm with the highest upper confidence bound, the learner, in LinUCB, selects the arm a_t such that:

$$a_t = \arg \max_{a \in \mathcal{A}_t} \left(\mathbf{x}_{t,a}^\top \hat{\theta}_a + \alpha \sqrt{\mathbf{x}_{t,a}^\top \mathbf{A}_a^{-1} \mathbf{x}_{t,a}} \right), \quad (16)$$

where the first term is the exploration reward and the second is the standard deviation of the expected reward presented in Equation (13), called the exploration bonus. α is the coefficient that balances the trade-off between exploration and exploitation:

- if α is too small: the decision-maker will choose the best arm several times, penalizing exploration;
- if α is too large: the learner explores most of the time, without exploiting the optimum arm.

In addition, $\mathbf{x}$ and $\mathbf{A}$ also influenced the balance. Unlike classic UCB, where the trade-off is based on t and N , in this case:

- $\mathbf{A}_a^{-1}$ represents the uncertainty around θ_a and accumulates information about the past of an arm. If an arm is tested many times, the matrix will be large, leading to a narrow confidence interval; conversely, there will be large uncertainty.
- $\mathbf{x}$ is related to the observed situation. It describes the condition in which the arm is tested. If the situation is new, the learner explores more; if it is already known, they tend to exploit it more.

Finally, for the selected arm θ parameters were updated as follows:

$$\begin{aligned} \mathbf{A}_{a_t} &\leftarrow \mathbf{A}_{a_t} + \mathbf{x}_{t,a_t} \mathbf{x}_{t,a_t}^\top, \\ \mathbf{b}_{a_t} &\leftarrow \mathbf{b}_{a_t} + r_t \mathbf{x}_{t,a_t}. \end{aligned} \quad (17)$$

In this work, a constrained version of LinUCB was implemented, similarly to the constrained UCB algorithm. First, both for accuracy and for time-in-air, the expected mean was calculated using Equation (13). They both have the same context $\mathbf{x}$ but different θ parameters. In each time step t , the algorithm calculates the $\tilde{\mathcal{A}}_{feas}$ subset as:

$$\tilde{\mathcal{A}}_{feas} \leftarrow \left\{ a_i \in \mathcal{A} \mid \mathbf{x}_{t,a}^\top \hat{\theta}_{a,eng} - \alpha \sqrt{\mathbf{x}_{t,a}^\top \mathbf{A}_{a,eng}^{-1} \mathbf{x}_{t,a}} < \gamma \right\}, \quad (18)$$

where γ is the same threshold θ used in the static problem and chosen as reported in Section 2.4.2. Equation (18) considers the LCB for the underestimation of the expected mean of the time-in-air feature to be optimistic over the engagement. From the new subset of $\mathcal{A}$, the learner selects the arm with the highest overestimation of the expected mean of accuracy, formally:

$$a_t = \arg \max_{a \in \tilde{\mathcal{A}}_{feas}} \left(\mathbf{x}_{t,a}^\top \hat{\theta}_{a,acc} + \alpha \sqrt{\mathbf{x}_{t,a}^\top \mathbf{A}_{a,acc}^{-1} \mathbf{x}_{t,a}} \right). \quad (19)$$

Once an arm was selected, $\hat{\theta}$ estimations are updated as in Equation (17).

To characterize the shape of the learning trajectory, session-level mean accuracy, i.e., the mean accuracy aggregated across children per within-child session number (with at least 5 children in the same session), was fitted against four candidate models: linear, power law, exponential plateau with free parameters, and constrained exponential plateau. The nonlinear least-squares method and model comparison based on R^2 identified the constrained exponential plateau as the best-fitting function ($R^2 = 0.719$). Accuracy was therefore modeled as $y(t) = 1 - e^{-t/\tau}$, constraining the asymptote to the theoretical maximum of the accuracy, i.e., 1, and the intercept to 0, with τ as the only free parameter estimated using nonlinear least squares. Parameter τ represents the number of sessions required to reach approximately 63% of maximum performance; full stabilization, 95%, is expected at 3τ sessions. Therefore, as context $\mathbf{x}$ was used for accuracy modeling, formally:

$$\mathbf{x}_t = 1 - e^{-t/\tau}. \quad (20)$$

Since a dynamic MAB is considered, it is reasonable to assume that the expected means of the reward and engagement increase rapidly at first, leading to a change in the time window. Then, when the last window is reached, the estimated averages stabilize at their values.

So, the pseudo-code of LinUCB is shown in Algorithm 5 .

Algorithm 5 LinUCB with constraint

Require: Input: A, θ, T, α

```

1: Initialize  $acc_{i,t=0}, eng_{i,t=0} \leftarrow 0$ 
2: Both acc and eng:  $\mathbf{A}_a \leftarrow \mathbf{I}_d$ 
3: Both acc and eng:  $\mathbf{b}_a \leftarrow \mathbf{0}_d$ 
4: for each  $a_i \in \mathcal{A}$  do
5:    $a_t \leftarrow a_i$ 
6:   Observe  $r_{a_t,t}, e_{a_t,t}$ 
7:   Update statistics of arm  $a_t$ 
8:    $t \leftarrow t + 1$ 
9: end for
10: for  $t = |\mathcal{A}| + 1$  to  $T$  do
11:   Both acc and eng:  $\hat{\theta} \leftarrow \mathbf{A}_a^{-1} \mathbf{b}_a$ 
12:    $\tilde{\mathcal{A}}_{feas} \leftarrow \left\{ a_i \in \mathcal{A} \mid \mathbf{x}_{t,a}^\top \hat{\theta}_{a,eng} - \alpha \sqrt{\mathbf{x}_{t,a}^\top \mathbf{A}_{a,eng}^{-1} \mathbf{x}_{t,a}} < \tau \right\}$ 
13:   if  $\tilde{\mathcal{A}}_{feas} = \emptyset$  then
14:     Choose  $a_t \in \mathcal{A}$ 
15:   else
16:      $a_t = \arg \max_{a \in \tilde{\mathcal{A}}_{feas}} \left( \mathbf{x}_{t,a}^\top \hat{\theta}_{a,acc} + \alpha \sqrt{\mathbf{x}_{t,a}^\top \mathbf{A}_{a,acc}^{-1} \mathbf{x}_{t,a}} \right)$ 
17:   end if
18:   Observe  $r_{a_t,t}, e_{a_t,t}$ 
19:   Both acc and eng:  $\mathbf{A}_{a_t} \leftarrow \mathbf{A}_{a_t} + \mathbf{x}_{t,a_t} \mathbf{x}_{t,a_t}^\top$ 
20:   Both acc and eng:  $\mathbf{b}_{a_t} \leftarrow \mathbf{b}_{a_t} + r_t \mathbf{x}_{t,a_t}$ 
21: end for

```

2.5.3 Evaluation Protocol

In the dynamic problem, only one configuration was taken into account. For the assessment of the optimum algorithm, the same approach described in Section 2.4.6 was used.

2.6. Deployment

To evaluate the effectiveness of the static framework in a real educational context, the selected algorithm can be implemented in the game. Having already trained the algorithm with real-world data, the model can be initialized with the final knowledge state. In more detail, for Thompson Sampling, the posterior distributions of both accuracy and time-in-air from the last step t become the new priors. For ϵ -greedy and UCB, the estimated mean reward and time-in-air are used to initialize the agent. From this setup, a future longitudinal study will be conducted to validate the system's impact by comparing an experimental group that plays the data-driven adaptive game with a control group that plays the standard version without the present framework. In this way, the comparison will allow to verify whether the developed framework enhances performance and sustains engagement over time.

3. Results

This section presents the main results of the thesis analysis and it is structured as follows: first, information of recruited participants and feature analysis were outlined, then, for both static and dynamic settings, the most significant results were reported.

3.1. Recruited Participants

66 first-grade children participated in the study. Their characteristics about age, sex, and handedness are reported in Table 7.

Total participants	Age mean $\pm$ std	Sex	Headedness
66	6.45 $\pm$ 0.31	31 F, 35 M	60 right, 6 left

Table 7: Participants information about age (mean $\pm$ standard deviation), sex, and handedness.

After cleaning, a total of 6058 sessions were considered, divided as reported in Table 8. Information regarding session quantity distribution is found in Table 9.

Total data	Easy	Medium	Hard
6058	749 (12%)	2052 (34%)	3257 (54%)

Table 8: Number of sessions per level.

Count	Mean	Std	Min	Q1	Median	Q3	Max
66	91.788	40.527	14	62.5	88	116.5	232

Table 9: Participants' information about how much they have played in Number Tracing.

3.2. Features Selection

In this part, the trends of accuracy and strokes ratio, the main performance metrics, are reported in Figure 4.

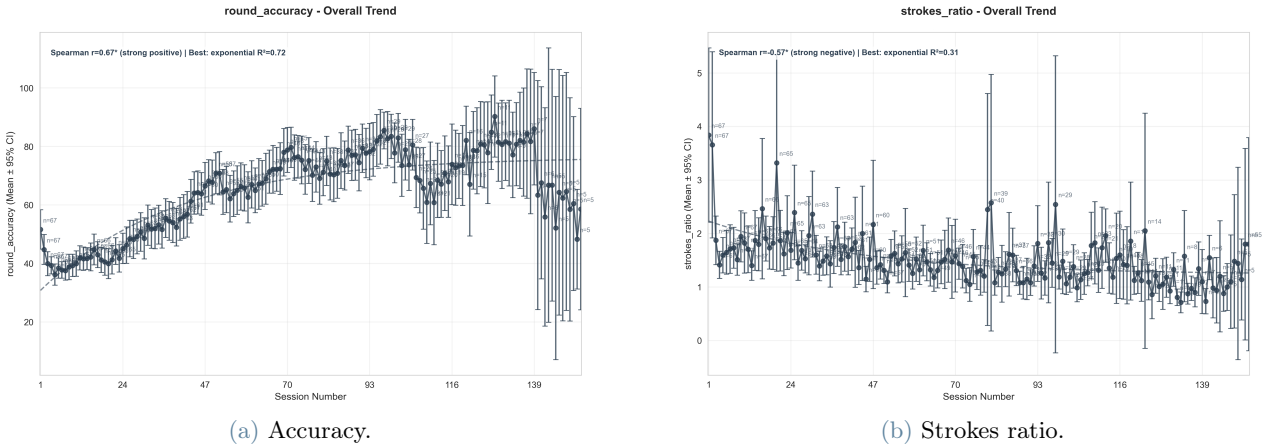


Figure 4: Features analysis (round accuracy and strokes ratio).

3.3. Threshold Selection

For threshold, the main statistical characteristics were calculated considering a total of 66 children and are shown in Table 10.

Count	Mean	Std	Min	Q1	Median	Q3	Max
6058	0.21540	0.21665	0.000	0.000	0.15835	0.35235	0.92620

Table 10: θ threshold statistical characteristics.

The third quartile Q3 value was selected as θ threshold. Therefore:

1. is based on the distribution of the observed data and is not an arbitrary value;
2. is robust to extreme values (outliers);
3. does not depend on parametric assumptions.

Moreover, Table 11 shows the θ threshold exceeding rates, i.e., how many times there is a $\mu_e(a)$ that violates the constraint imposed by the established threshold, for each difficulty level. This helps to visualize how the chosen value is reflected in each configuration. Note that, for individual configuration, some students have not played some levels.

Configuration	Easy	Medium	Hard
Individual	5/43 (11.6%)	22/62 (35.5%)	2/63 (3.2%)
Cluster	0/2	1/2 (50%)	0/2
Aggregated	0/1	0/1	0/1

Table 11: For each configuration, are reported how many times an arm exceed the threshold over the total number of occurrences.

3.4. Results of Static Simulation

Finally, once a value of θ was chosen, simulations were performed for all configurations with a time horizon $T = 10,000$ for 10 runs, whose calculated CI is 95%. As already discussed in Section 2.4.5, in each configuration, the average mean reward, cumulative regret, and arm selection frequency were calculated and plotted. Referring to ϵ -greedy, two different variants were tested: the first has $\epsilon = 0.1$, the second has $\epsilon = 0.2$.

3.4.1 Individual Configuration

In individual configuration, a total of 630 simulations ($n_{child} \cdot n_{runs}$) were performed. This section reports the most representative results that illustrate the algorithms' general trends in possible real-world cases.

1. Child 44: whose results represent the standard case in which all arms are feasible. Algorithms have their standard behavior.
2. Child 48: whose results show the typical behavior of algorithms when there is at least one unfeasible arm and $\mu_e(a)$ is far from the threshold.
3. Child 3: results represent a case in which μ_e of an arm is just above the threshold.
4. Child 1: a case in which all available arms are feasible, but one of them has μ_e very narrow to the thresholds.
5. Child 60: an example of what happens when no arm is feasible.

Results for Child 44 are shown in Figures 5 and 6. This is representative of the standard case in which all arms are feasible, because they have $\mu_e(a) \leq \theta$.

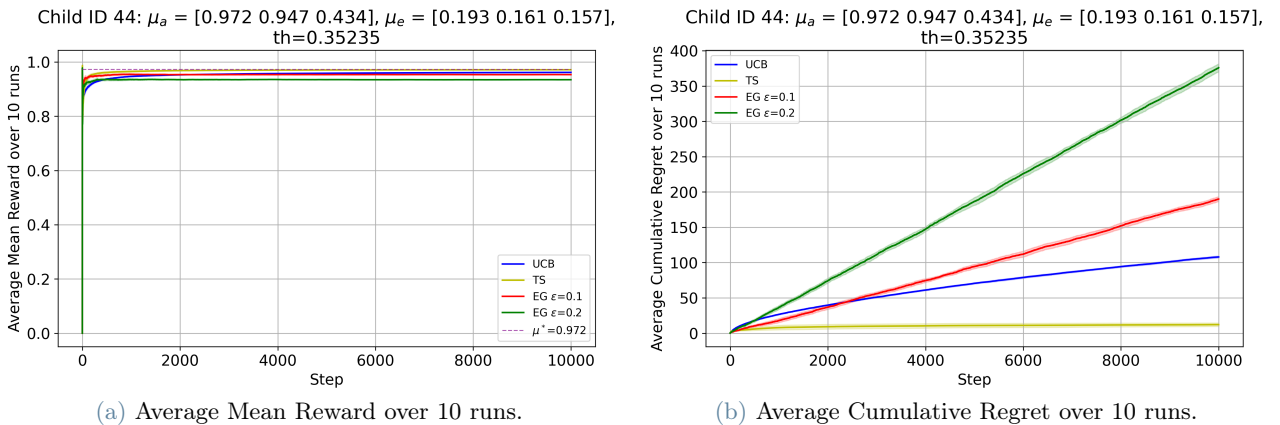


Figure 5: Average mean reward (a) and Average cumulative regret (b) over 10 runs for child 44. In Figure (a) dotted line represents the μ^* trend. In both: UCB is in blue, Thompson Sampling in yellow, ϵ -greedy (0.1) in red, and ϵ -greedy (0.2) in green.

Results for Child 48 are reported in Figures 7 and 8. This case is representative of the constrained case in which an unfeasible arm has its μ_e far from the threshold, so the optimal arm becomes the sub-optimal one

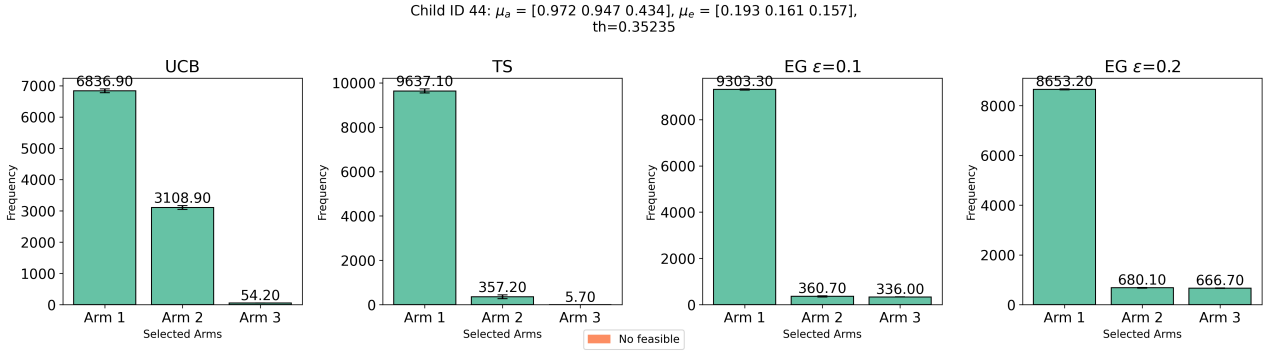


Figure 6: Arm selection frequency for child 44. Green represents feasible arms and orange represents unfeasible arms.

immediately after, from an accuracy point of view.

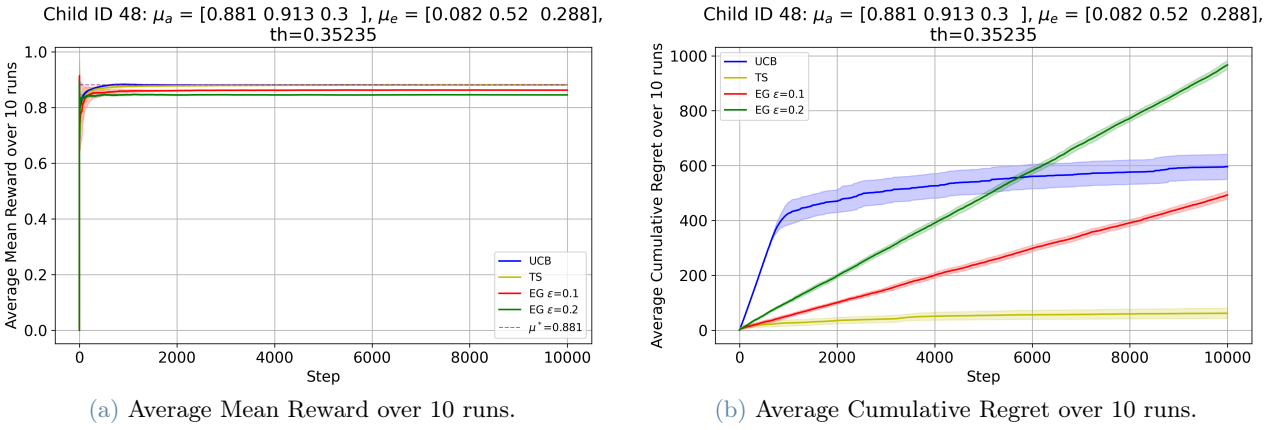


Figure 7: Average mean reward (a) and Average cumulative regret (b) over 10 runs for child 48. In Figure (a) dotted line represents the μ^* trend. In both: UCB is in blue, Thompson Sampling in yellow, ϵ -greedy (0.1) in red, and ϵ -greedy (0.2) in green.

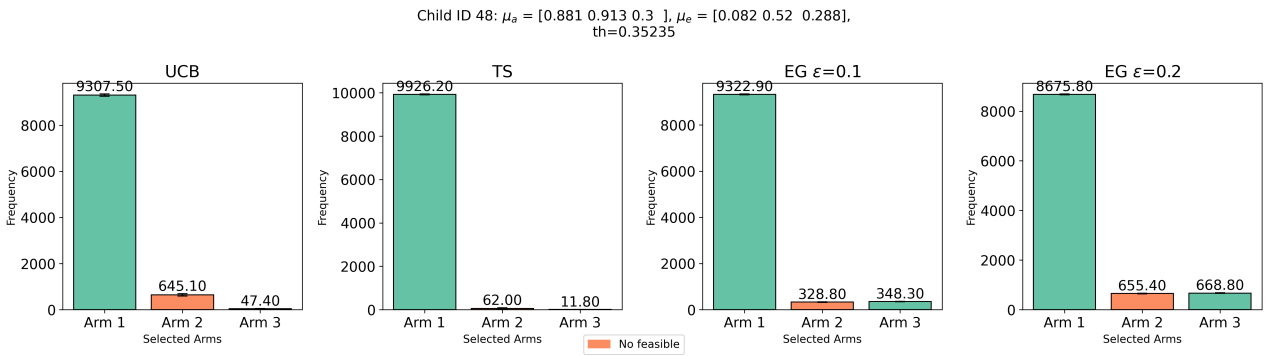
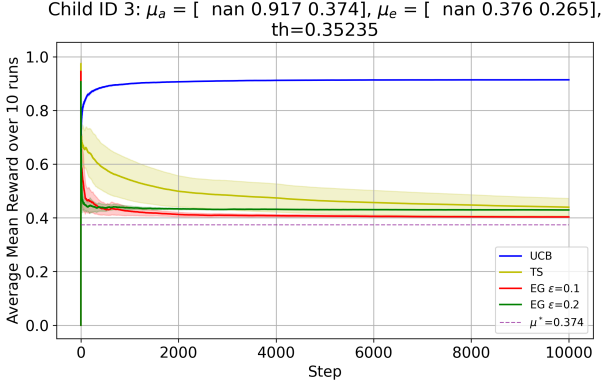


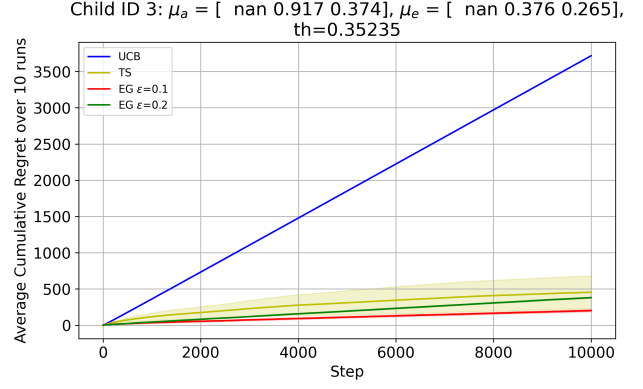
Figure 8: Arm selection frequency for child 48. Green represents feasible arms and orange represents unfeasible arms.

Results for Child 3 are reported in Figures 9 and 10. The second arm is not considered feasible because its μ_e exceeds the threshold. The optimum arm is the third.

Results for Child 1 are reported in Figures 11 and 12. As shown in the figures, all arms are feasible, but one of them has μ_e very close to the threshold.



(a) Average Mean Reward over 10 runs.



(b) Average Cumulative Regret over 10 runs.

Figure 9: Average mean reward (a) and Average cumulative regret (b) over 10 runs for child 3. In Figure (a) dotted line represents the μ^* trend. In both: UCB is in blue, Thompson Sampling in yellow, ϵ -greedy (0.1) in red and ϵ -greedy (0.2) in green.

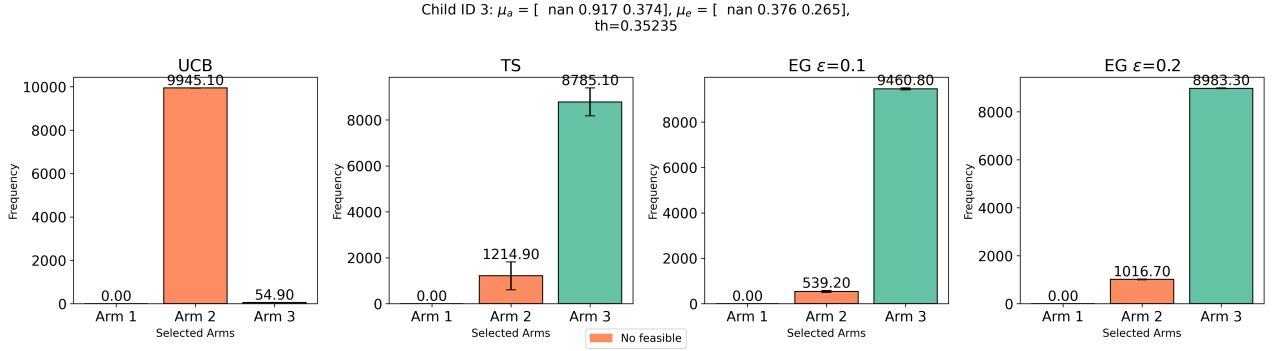
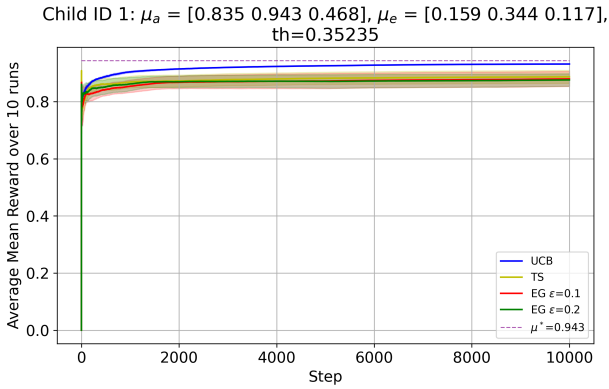
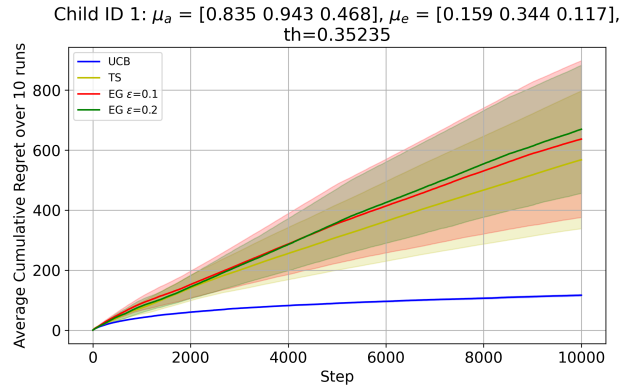


Figure 10: Arm selection frequency for child 3. Green represents feasible arms and orange represents unfeasible arms.



(a) Average Mean Reward over 10 runs.



(b) Average Cumulative Regret over 10 runs.

Figure 11: Average mean reward (a) and Average cumulative regret (b) over 10 runs for child 18. In Figure (a) dotted line represents the μ^* trend. In both: UCB is in blue, Thompson Sampling in yellow, ϵ -greedy (0.1) in red, and ϵ -greedy (0.2) in green.

Results for Child 60 are reported in Figures 13 and 14. This example shows the behavior of all algorithms in a situation in which all the $\mu_e(a)$ exceed the θ threshold.

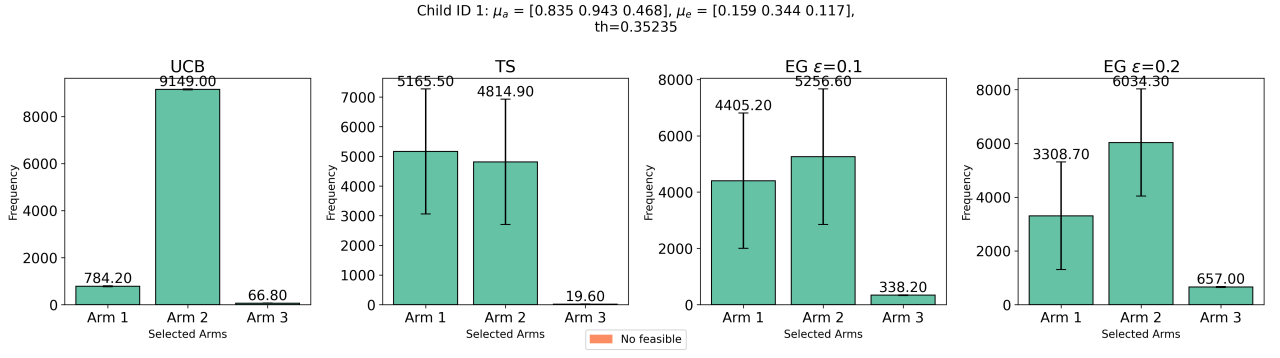


Figure 12: Arm selection frequency for child 1. Green represents feasible arms and orange represents unfeasible arms.

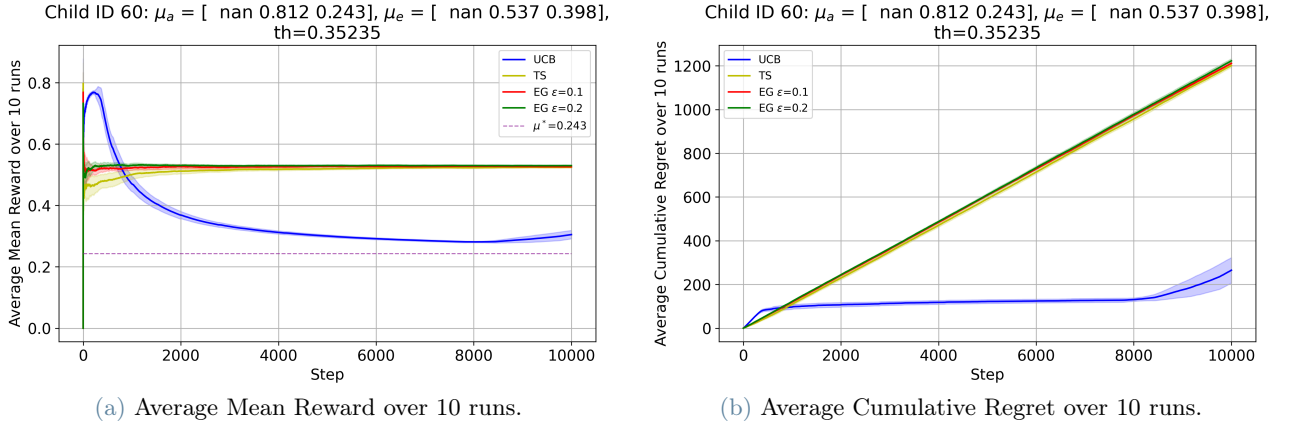


Figure 13: Average mean reward (a) and Average cumulative regret (b) over 10 runs for child 60. In Figure (a) dotted line represents the μ^* trend. In both: UCB is in blue, Thompson Sampling in yellow, ϵ -greedy (0.1) in red, and ϵ -greedy (0.2) in green.

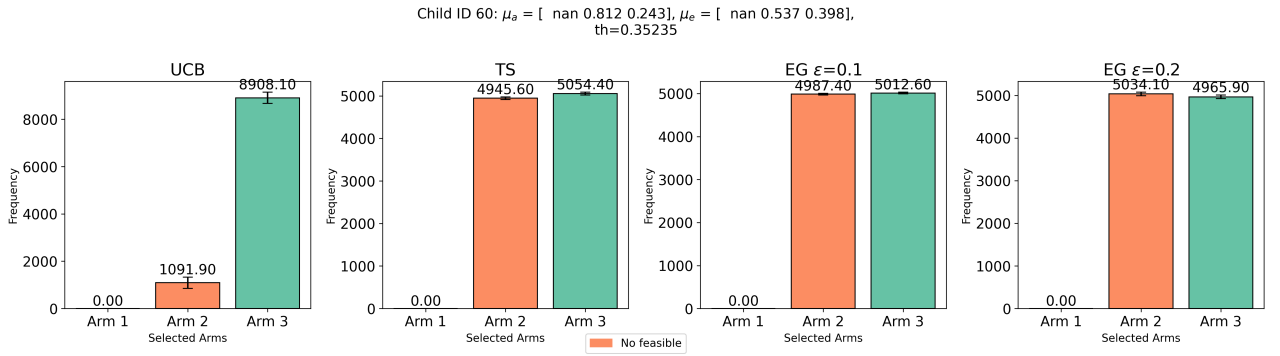
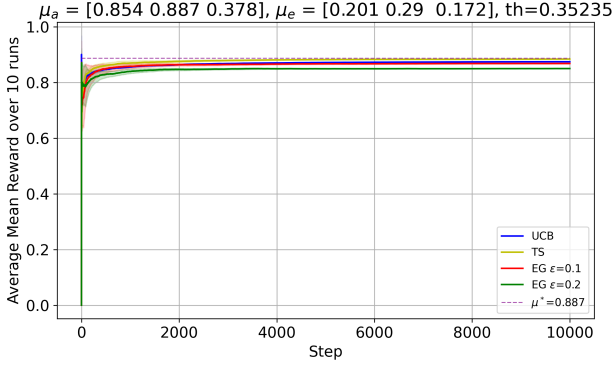


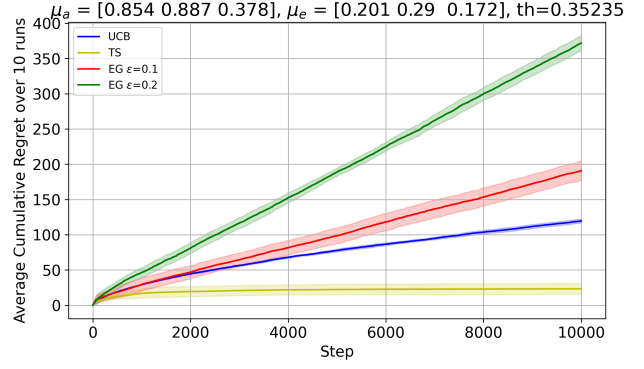
Figure 14: Arm selection frequency for child 60. Green represents feasible arms and orange represents unfeasible arms.

3.4.2 Aggregated

For the aggregated configuration, 10 runs were performed, and the results are reported in Figures 15 and 16. No arm has μ_e above the thresholds. It can be considered a standard MAB case.



(a) Average Mean Reward over 10 runs.



(b) Average Cumulative Regret over 10 runs.

Figure 15: Average mean reward (a) and Average cumulative regret (b) over 10 runs for aggregated configuration. In Figure (a) dotted line represents the μ^* trend. In both: UCB is in blue, Thompson Sampling in yellow, ϵ -greedy (0.1) in red, and ϵ -greedy (0.2) in green.

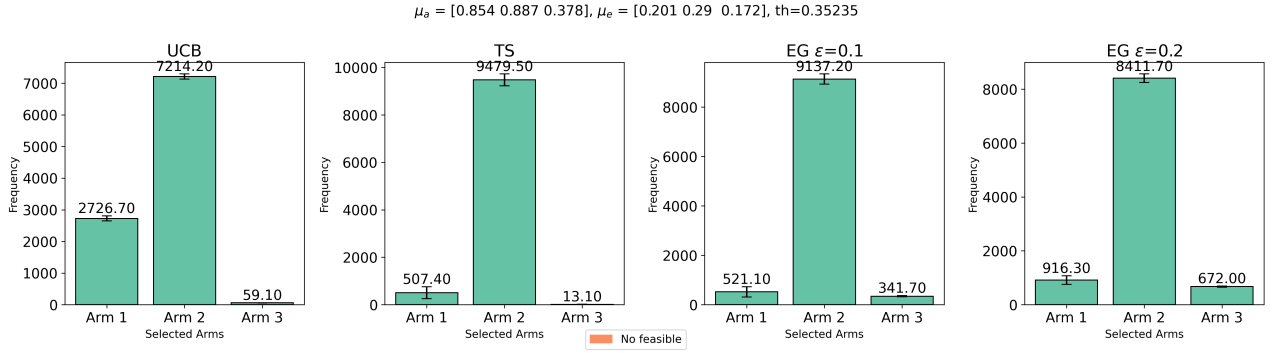
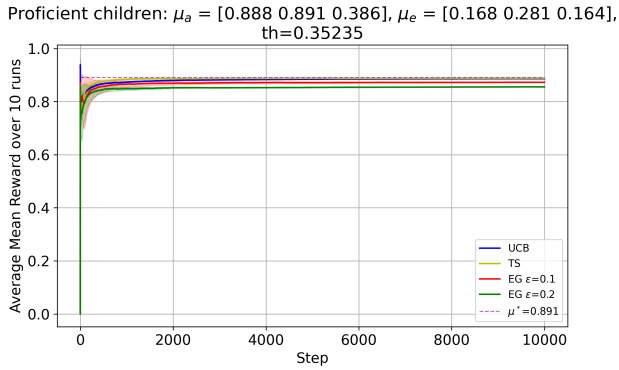


Figure 16: Arm selection frequency for aggregated configuration. Green represents feasible arms and orange represents unfeasible arms.

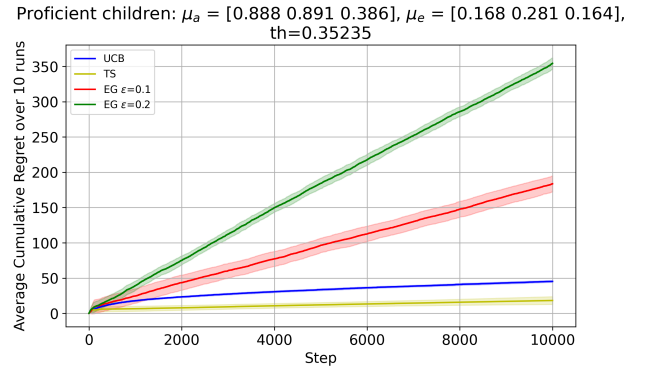
3.4.3 Cluster

In this subsection, all the results for clusters are presented. In particular, 20 simulations were run.

Results for Proficient children are reported in Figures 17 and 18. All arms can be considered feasible.



(a) Average Mean Reward over 10 runs.



(b) Average Cumulative Regret over 10 runs.

Figure 17: Average mean reward (a) and Average cumulative regret (b) over 10 runs for cluster 0. In Figure (a) dotted line represents the μ^* trend. In both: UCB is in blue, Thompson Sampling in yellow, ϵ -greedy (0.1) in red, and ϵ -greedy (0.2) in green.

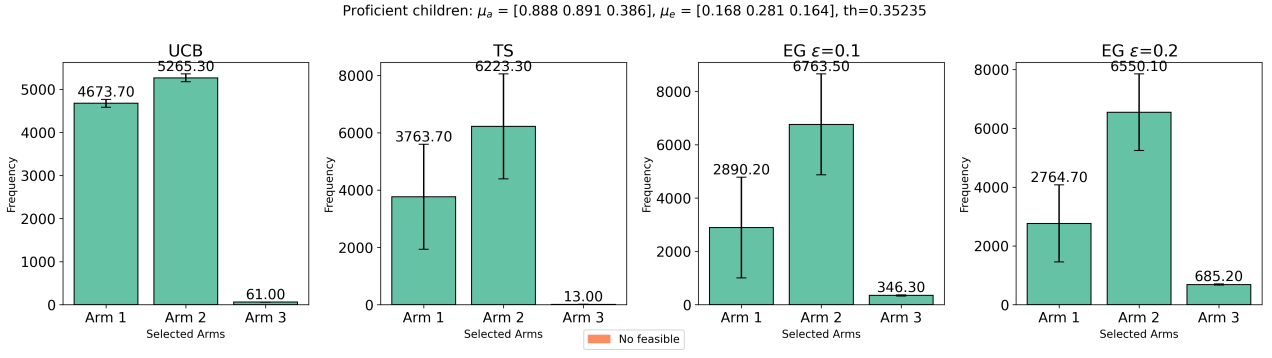


Figure 18: Arm selection frequency for proficient children cluster. Green represents feasible arms and orange represents unfeasible arms.

Results for Struggling Children are shown in Figures 19 and 20. The medium arm is unfeasible, so the optimal action is the first.

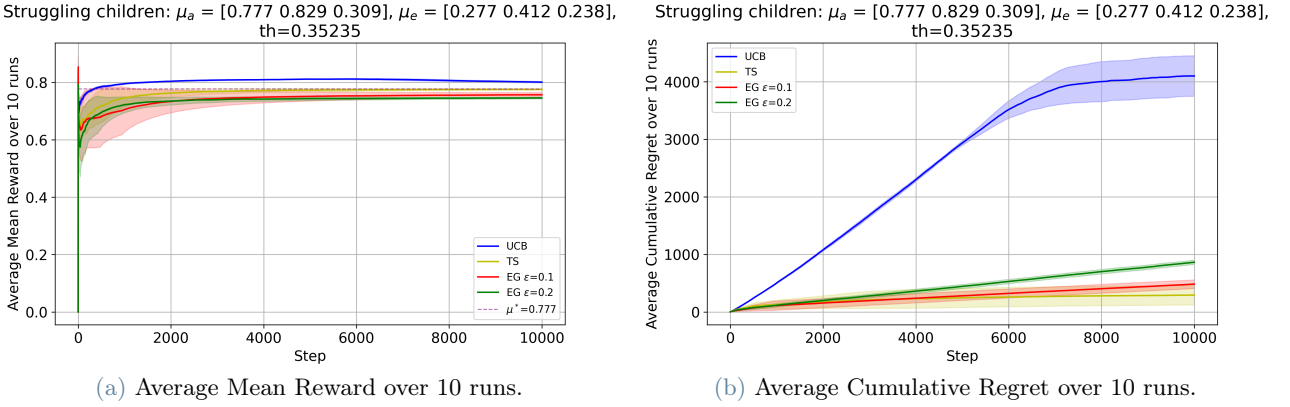


Figure 19: Average mean reward (a) and Average cumulative regret (b) over 10 runs for cluster 1. In Figure (a) dotted line represents the μ^* trend. In both: UCB is in blue, Thompson Sampling in yellow, ϵ -greedy (0.1) in red, and ϵ -greedy (0.2) in green.

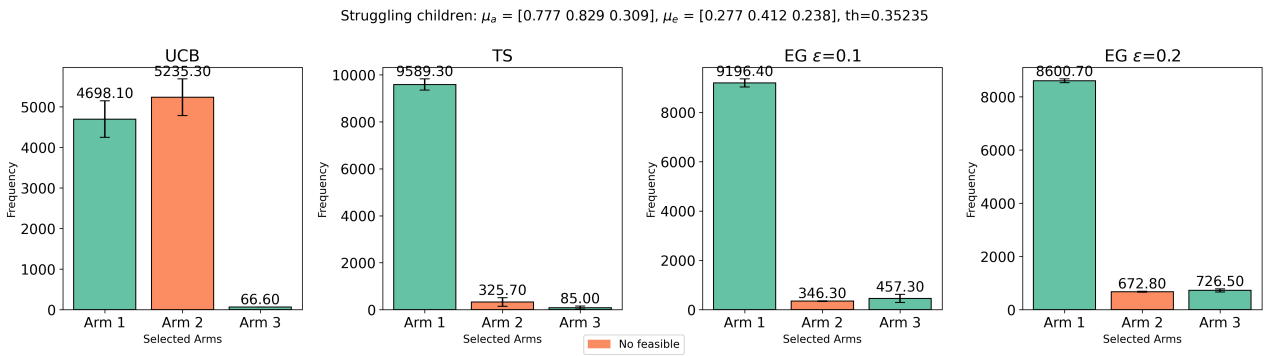


Figure 20: Arm selection frequency for struggling children cluster. Green represents feasible arms and orange represents unfeasible arms.

3.4.4 Best Algorithm and Configuration

As anticipated in Section 2.4.6, for each algorithm and each configuration was taking into account the mean and the 95% confidence interval of the sum of cumulative reward and cumulative regret for each run. The results are reported in Table 12.

Configuration	Algorithm	Cumulative Reward	Cumulative Regret
<i>Individual</i>	UCB	4879.274 $\pm$ 4.480	1097.314 $\pm$ 2.622
	TS	4628.369 $\pm$ 41.557	751.457 $\pm$ 46.769
	EG $\epsilon = 0.1$	4511.050 $\pm$ 63.566	563.560 $\pm$ 57.982
	EG $\epsilon = 0.2$	4490.201 $\pm$ 55.693	670.822 $\pm$ 49.706
<i>Aggregated</i>	UCB	5287.151 $\pm$ 5.657	87.041 $\pm$ 1.966
	TS	5348.487 $\pm$ 9.325	22.567 $\pm$ 7.545
	EG $\epsilon = 0.1$	5253.090 $\pm$ 14.454	118.956 $\pm$ 12.306
	EG $\epsilon = 0.2$	5144.632 $\pm$ 7.909	227.596 $\pm$ 6.388
<i>Cluster</i>	UCB	5290.492 $\pm$ 4.019	374.296 $\pm$ 6.019
	TS	5269.123 $\pm$ 32.196	125.735 $\pm$ 59.998
	EG $\epsilon = 0.1$	5154.716 $\pm$ 64.146	190.684 $\pm$ 62.160
	EG $\epsilon = 0.2$	5070.311 $\pm$ 32.896	286.950 $\pm$ 34.621

Table 12: Mean and 95% confidence intervals for both cumulative reward and cumulative regret. The highlighted cells represent the best pair of configuration and algorithm.

3.5. Results of Dynamic Simulation

As in the static case, to evaluate the optimal algorithm that best suits the problem’s modeling, the mean reward, cumulative reward, cumulative regret, and arm selection frequency were used as performance metrics. As already explained in Section 2.5.1, each window comprises at least 30 samples for each difficulty level, for a total of 17 windows. In Figure 13, the exceeding rate of the established threshold, over the total window, for each level, is reported. Note that in the last time window, no easy-level data is available.

Difficulty	Unfeasible arm	Total window	Ratio (%)
Easy	0	16	0
Medium	2	17	11.76%
Hard	0	17	0

Table 13: Counts on how many times each level exceeds the threshold in each time window.

The average mean reward and cumulative regret over 10 runs are presented in Figure 21 with their 95% confidence interval along 10,000 time steps, with the same θ of the static case. In Figure 21a, the dotted line represents the μ^* identified in order to better understand the ideal μ that algorithms should converge to. Referring to LinUCB, its results are obtained using $\tau = 21$, chosen fitting of the data as explained in Section 2.5.2, and an α parameter equal to 1 to balance the exploration-exploitation trade-off.

Figure 22 shows how much each algorithm has selected an arm. The average frequency and its 95% confidence interval are taken into account.

3.5.1 Best Algorithm

Finally, to numerically identify the optimal behavior, the cumulative reward sums were averaged over 10 runs, as in the static case. The results are reported in Table 14, with the best value highlighted.

Configuration	Algorithm	Cumulative Reward	Cumulative Regret
<i>Aggregated dynamic</i>	UCB	2327.360 $\pm$ 3.216	217.754 $\pm$ 12.555
	TS	2338.153 $\pm$ 13.769	195.511 $\pm$ 39.240
	EG $\epsilon = 0.1$	2284.666 $\pm$ 12.769	221.383 $\pm$ 49.410
	EG $\epsilon = 0.2$	2260.998 $\pm$ 13.074	280.745 $\pm$ 28.088
	LinUCB	2360.033 $\pm$ 4.544	181.455 $\pm$ 24.287

Table 14: Mean and 95% confidence intervals for both cumulative reward and cumulative regret. The highlighted cells represent the best algorithm.

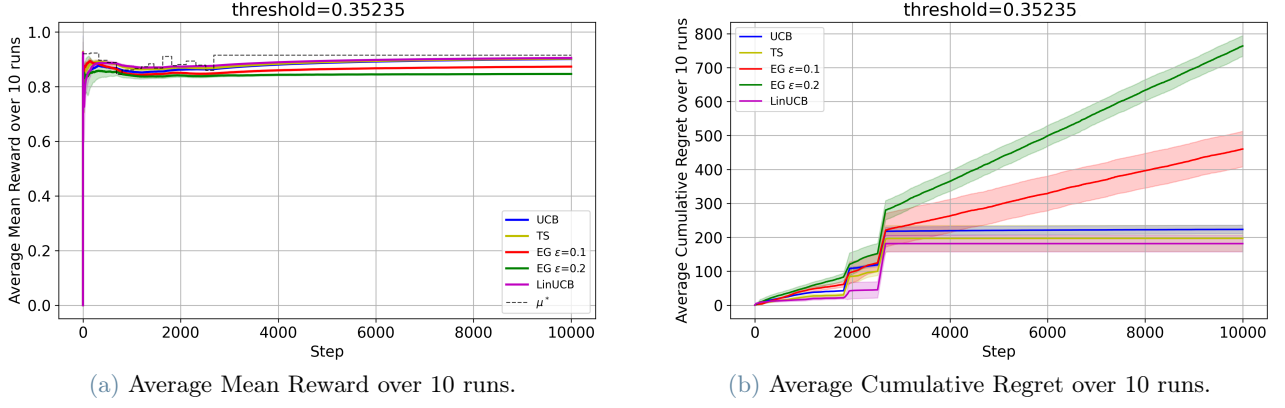


Figure 21: Average mean reward (a) and Average cumulative regret (b) over 10 runs. In Figure (a), the dotted line represents the μ^* trends over time, reflecting the change of the time window. In both: UCB is in blue, Thompson Sampling in yellow, ϵ -greedy (0.1) in red, ϵ -greedy (0.2) in green, and LinUCB in magenta.

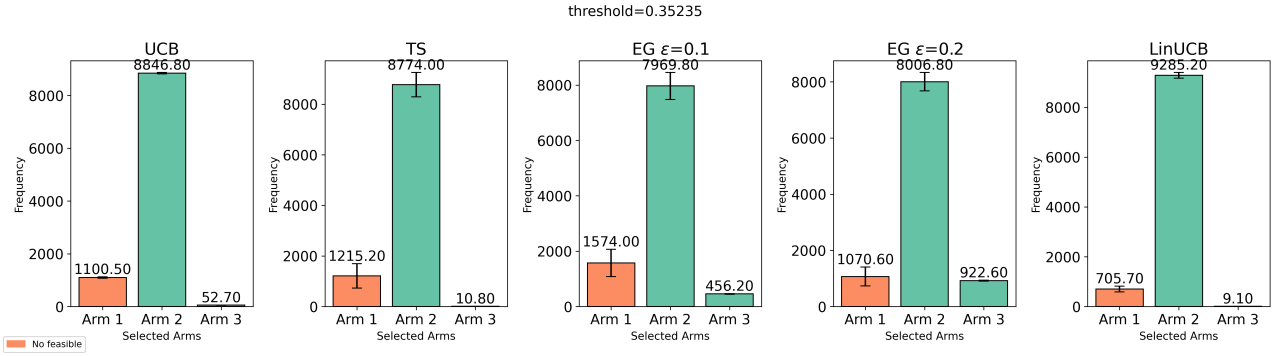


Figure 22: Arm selection frequency in the dynamic approach. Green represents feasible arms and orange represents unfeasible arms in the last time window, where data for the easy level are not available.

3.6. Deployment

Having identified Thompson Sampling in an aggregated configuration as the best pair, the algorithm can be implemented to evaluate its effectiveness. As explained in Section 2.6, posterior distributions, whose example is reported in Figure 23, can be used as new priors.

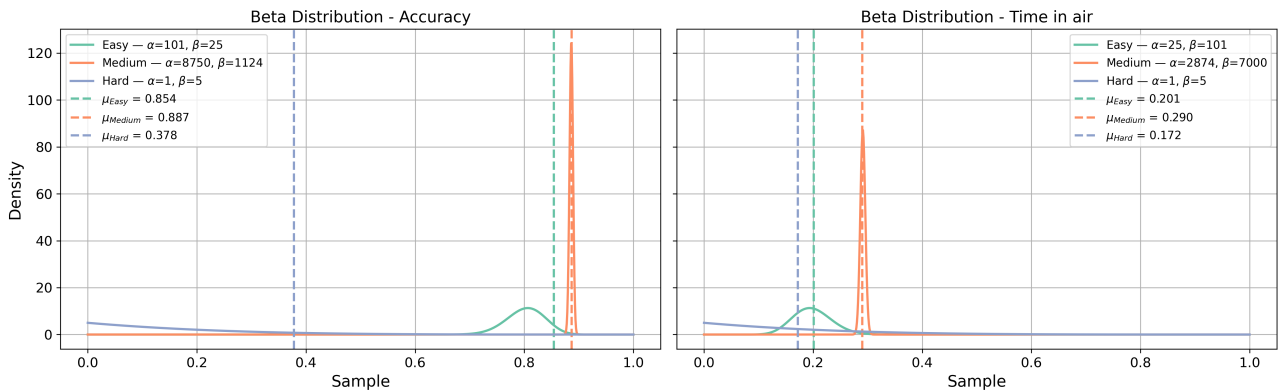


Figure 23: An example of a new prior distribution based on the final beta distributions of one static aggregated configuration run, for both accuracy and time-in-air for each level.

4. Discussion

4.1. Recruited Participants

In general, students have played the Number Tracing serious game at around 92 on average, with one student reaching a peak of 232 (see Table 9). This led to the understanding that, in a child-specific configuration, the available data for each student is insufficient to test the algorithms. Moreover, the distribution of data across the three difficulty levels is not homogeneous, with more than half of the data at the hard level. Therefore, not only are there few data per child, but there is also a higher prevalence of third level data. In an evaluation protocol that uses uniform random resampling in Section 2.4.6, data distributions affect results. Using a different configuration allows to consider more data in respect to individual one.

4.2. Features Selection

From a previous work, performance metrics were analyzed. In particular, in Number Tracing emerged that *accuracy*, related to the tracing part, increased, and *strokes ratio* decreased. Compared to the first attempts, the last valid session shows a more accurate execution, as can be seen from the trends in the graphs 4. In particular, accuracy, the main performance metric, increased from 44.44% to 52.12% and strokes ratio decreased from 2.0 to 1.0. This results validate the use of accuracy features as reward for the MAB algorithms, since it effectively captures the user’s learning trajectories and performance enhancement.

4.3. Threshold Selection

Once a better understanding of the children’s playing characteristics, in relation to what is discussed in Section 3.4, the results for each configuration are reported in a static case. First, as shown in Table 10, the 75th percentile was chosen as the threshold value. Considering this amount, it directly affects which level can be considered feasible. With this θ , the level with the most unfeasible cases is medium, followed by the easy one, and the configuration with the most cases is the individual. Cluster and aggregated configurations, which take into account more data together compared to child-specific, have lower averages $\mu_e(a)$, leading to fewer unfeasible arms in these setups.

4.4. Static Simulation

4.4.1 Individual Configuration

For individual configurations, different cases are reported, including standard MAB, constrained with $\mu_e(a)$ far and near the threshold, and fallback.

All arms are considered feasible as long as their real mean of time-in-air does not exceed the threshold. The first example (Section 3.4.1) is representative of the standard behavior of MAB methods, in which the optimal arm is the one with the highest accuracy (reward) value. In more detail, considering the ordered arm (easy, medium, and hard), their real mean of time-in-air is: $\mu_e = [0.193, 0.161, 0.157] \leq \theta$, consequently μ^* becomes the maximum $\mu_a(a)$ and therefore $\mu^* = \max [0.972, 0.947, 0.434] = 0.972$. Starting to analyze algorithm behaviors, ϵ -greedy generally has an average reward very close to μ^* from the first steps. The closeness depends on the ϵ probability. A lower ϵ tends to favor the exploitation of the optimal arm over the exploration of the less known arms, from the very first steps. Testing it with two different ϵ values allows to understand how its performance varies as the exploration probability increases. In this sense, as can also be seen in the graph 5a, EG with $\epsilon = 0.1$ has the highest average mean reward in the first steps, even compared to EG with $\epsilon = 0.2$, which tends to explore with a 10% higher probability. Consequently, however, ϵ -greedy also has a regret (Figure 5b) that grows linearly over time. As explained in Section 2.4.3, this depends on the fact that it continues to explore over time, along time horizon T , with the same probability. The higher the epsilon, the greater the regret. Secondly, UCB has a lower average reward than ϵ -greedy in the early stages. This behavior is caused by the confidence bounds: initially, all arms have high uncertainty, so this method explores much more than ϵ -greedy. Over time, precisely because the confidence bounds decrease, UCB gets much closer to μ^* , selecting the best arm many more times. This behavior is also reflected in regret. It no longer grows linearly but has a logarithmic trend: it grows rapidly at the beginning, because it explores so much due to larger confidence bounds for all arms, and slows down over the time horizon. Finally, Thompson Sampling performs best. In terms of reward, it actually has the worst average reward at the beginning; in fact, it explores much more than ϵ -greedy. Since each arm is associated with a beta distribution, whose prior is the uniform probability distribution, each sample, both in

terms of accuracy and time-in-air, initially has the same probability of being drawn. As the time steps progress, increasingly more precise estimates will be obtained, and the average mean reward will tend to μ^* , even more than UCB. Also, in the regret graph, it is significantly inferior to UCB, growing rapidly at the beginning but stabilizing in the first hundred steps. As an additional performance metric, the same trends are evident in Figure 6. It can be seen that each algorithm selects the optimal arm many more times than the others do. Thompson Sampling almost never selects the worst arm, but often selects the second-highest difficulty level because the two are very close in accuracy. The same behavior can be observed with UCB. On the other hand, ϵ -greedy, always exploring with the same probability, selects the third arm many times, with EG with $\epsilon = 0.2$ selecting both sub-optimal arms twice as often in respect to EG with $\epsilon = 0.1$. Finally, among confidence intervals, UCB is the best. This difference can be explained by the fact that it is a deterministic decision-making method. Both ϵ -greedy and Thompson Sampling are stochastic from this perspective: they have wider confidence intervals.

Another case is the situation presented in Section 3.4.1. This has been reported because it represents the constrained version. Looking at the time-in-air means, the medium level significantly exceeds the threshold: $\mu_e = [0.082, 0.520, 0.288]$. Therefore, the optimum arm becomes the first level: $\mu_a = [0.881, 0.913, 0.300]$ with $\mu^* = 0.881$. In terms of mean reward, it differs little from the previous case. The big difference can be seen in regret (graph 8): given how it is defined, i.e., with a penalty when an unfeasible arm is selected, it is clear that UCB, selecting the second level many times, has a very high regret initially that linearly increases. Over time horizon T , the same can be seen in both ϵ -greedy: by randomly choosing the second arm many times, it tends to have a much higher regret. For example, EG $\epsilon = 0.2$ has almost a regret of 1000 compared to the previous case, where it was around 400. Hence, it can be assumed that, in this case too, Thompson Sampling performs best both in terms of average mean reward and cumulative regret, choosing the unfeasible arm very rarely. Despite the very high regret, which reflects only the choice of unfeasible arms, all three algorithms identify the first arm as the best, choosing the second arm rarely, which, in terms of accuracy, would yield a higher payoff. Therefore, all three can obtain a correct estimate of the time-in-air. In terms of variability, only UCB differs from run to run, especially in regret. This probability arises because, as the number of steps increases, UCB occasionally pulls the second arm instead of the first.

The results presented in Section 3.4.1 let to see a situation in which the second arm is the optimum in terms of accuracy but has a μ_e that exceeds the thresholds: $\mu_a = [-, 0.917, 0.374]$ and $\mu_e = [-, 0.376, 0.265]$. The best action is, therefore, the third: $\mu^* = 0.374$. In this situation, the two ϵ -greedy algorithms perform best, essentially because they exploit with a probability of $1 - \epsilon$. Thompson Sampling correctly identifies the best arm even though it also selects the second arm around 1,000 times. This is reflected in lower performance than ϵ -greedy, both in terms of average mean reward and cumulative regret. The algorithm with the worst behavior is UCB. Since the second arm has a mean of the engagement μ_e relatively close to the θ threshold and a mean of the accuracy μ_a significantly higher than the third level, once it reaches the point at which it is able to determine that the arm is not feasible, it actually returns feasible again after a few steps. Given how confidence intervals are defined, with the natural logarithm of the time step t in the numerator, they widen again when UCB stops selecting this arm. For these reasons, it almost exclusively selects the best action solely on the basis of accuracy. From a variability point of view, only Thompson Sampling appears to be more variable than the others, which, as written before, depends on the sampling of the beta distribution and on the proximity of the mean of the time-in-air of the second arm to the threshold.

There is a particular case in which the arm with the highest mean of accuracy $\mu_a = [0.835, 0.943, 0.468]$ is also the one with the mean of time-in-air $\mu_e = [0.159, 0.344, 0.117]$ very close to the threshold but is still considered feasible in Section 3.4.1: so $\mu^* = 0.943$. In this example, UCB is the best in terms of mean reward, cumulative regret, and selection frequency, and with very low run-to-run variability. The other two algorithms, besides being highly variable due to their stochasticity, also frequently select the third arm. If ϵ -greedy estimates a mean $\hat{\mu}_e$ above the threshold, that arm will no longer be chosen unless it is randomly pulled with ϵ probability. Even so, obtaining stochastic reward from the environment may not reduce the estimated mean enough to become feasible again. On the other hand, Thompson Sampling's low performance depends on the mean μ_e proximity of the second arm to the threshold. By sampling the associated beta distribution, it often obtains values above θ . This significantly affects results: the algorithm tends to be highly variable and selects the least accurate arm.

Finally, the last example reported is shown in Section 3.4.1. The three arm have $\mu_a = [-, 0.812, 0.243]$ and $\mu_e [-, 0.537, 0.398]$. Exceeding all of them by the threshold, the optimum arm becomes the third, with $\mu^* = 0.243$. In a similar situation, the best arm is identified as the one with the lowest μ_e , prioritizing engagement over accuracy, as explained in Section 2.4.2. The selection strategy remains the same, but when $\hat{A}_{feas} = \emptyset$, they start to randomly select. Thompson Sampling and ϵ -greedy tend to converge to the average of the two available arms. Probably, they correctly estimate the mean at the second level and exhibit some oscillation at the third level because it is near the threshold. Despite that, when both have, at some point, no feasible arms available, they

start to randomly choose. Again, UCB behaves differently from the others. As can be seen from the average mean reward in Figure 13, initially, all the confidence bounds are large, so it chooses the first one most of the time, because it has a higher payoff. When it understands that the arm is not feasible, it selects the third one. UCB is also the only algorithm with a larger confidence interval, especially at the end. This trend is probably also caused by the correct estimation of μ_e for the third arm. When UCB understands that this arm is also not feasible, it starts pulling randomly, as the others do. Moreover, since the third-level engagement mean is very close to the threshold, when UCB stops selecting it, the hard level becomes feasible again. Hence, in this case study, the combination of different effects, including the proximity to threshold, the empty feasible arm subset, and the intrinsic stochasticity/determinism of the algorithms, influence the results.

4.4.2 Aggregated Configuration

The results about the aggregated configuration can be consulted in Section 3.4.2. Across the entire dataset, all arms have engagement means $\mu_e = [0.201, 0.29, 0.172]$ below the threshold θ : they can be considered feasible. This is no different from the standard MAB problem. In fact, the second arm is the best in terms of accuracy, and therefore reward, since $\mu_a = [0.854, 0.887, 0.378]$, being selected by all three at most steps. Only UCB often selects the first, likely because the accuracy means are very close. Even in terms of variability, they are stable, though UCB performs better.

4.4.3 Cluster Configuration

For the cluster configuration, the results for both clusters are reported in Section 3.4.3. In the proficient children cluster, all arms are identified as feasible, since $\mu_e[0.168, 0.281, 0.164] \leq \theta$, as shown in 3.4.3. Then, considering $\mu_a = [0.888, 0.891, 0.386]$, $\mu^* = 0.891$. Although this case is similar to the one presented in the individual configuration, in which all arms were considered feasible (Section 3.4.1), the behaviors differ slightly. This essentially depends on the mean of the accuracy μ_a for the first and second level of difficulty, which differ by $\Delta = 0.003$. In fact, all three methods select both levels in an almost equivalent way, with UCB being less variable because of its deterministic choices. Thompson Sampling and ϵ -greedy have larger confidence intervals in all three graphs. Despite these considerations about variability, Thompson Sampling performs best. In Figure 17, it has the highest mean reward and also a very low cumulative regret over the 10 runs.

In the struggling children cluster, on the other hand, the second arm is identified as unfeasible, since $\mu_e = [0.277, 0.412, 0.238]$, even though it would be the best in terms of performance and accuracy, because $\mu_a = [0.777, 0.829, 0.309]$. Therefore, $\mu^* = 0.777$. Despite μ_e not being very close to θ , UCB still struggles to identify which arm is the best, repeatedly selecting the second, which offers a higher reward. As shown at the end of the cumulative regret in Figure 19b, the linear trend appears to be flattening and unstable. Presumably, by extending the time horizon T , it would be able to reach a plateau and stop selecting the second arm. For these reasons, its performance is highly variable, especially when starting to correctly estimate the means at the medium level. ϵ -greedy and Thompson Sampling show similar behavior as already explained in relation to graphs 3.4.1. The first almost always explores suboptimal arms only during the exploration phase; the second selects the unfeasible arm a few hundred times due to the higher reward; and the third almost never does.

4.4.4 Best Algorithm

In Section 3.4.4, Table 12 reports the mean and confidence intervals of the sum of cumulative reward and cumulative regret for each run, considering a time step t related to the number of sessions really played by the student. The highest cumulative reward and the lowest cumulative regret are achieved by Thompson Sampling in the aggregated configuration. These results remain superior, even when confidence intervals are considered, as they do not overlap with those of the other algorithms. Thompson Sampling, in general, reaches μ^* faster than other algorithms and has logarithmic regret. The fact that the aggregated configuration achieves such a high cumulative reward depends not only on the amount of data per simulation considered, which was significantly higher than the other two, but also on how far $\mu_e(a)$ is from the threshold. In the individual configuration, more unfeasible arms were present than in the other two. In particular, since all arms are feasible in an aggregated one, all the algorithms performed as well as the unconstrained version.

Note that, since cumulative regret is penalized as (6), the mean and the confidence intervals considered could be distorted. It is reasonable to look at cumulative reward as the primary metric and consider regret secondarily.

4.5. Dynamic Simulation

In a dynamic context, where the amount of data considered is less than half the original dataset, and the data are grouped into a time window, with the same θ threshold as in the static case, only two medium levels are considered unfeasible.

Analyzing the metric performance (Figure 21 and 22), LinUCB performs best in all of them. On average, the mean reward converges to μ^* fastest, with respect to the other three. Also, UCB and Thompson Sampling converge to the optimal solution over the time horizon T , but more slowly than LinUCB. On the other hand, both ϵ -greedy have lower performance in terms of mean reward, with EG $\epsilon = 0.2$ having the worst behavior. Regarding the regret, in the first two thousand steps, the typical trend can be recognized: Thompson Sampling and UCB have a logarithmic increase, whereas both ϵ -greedy linearly increase. When time advances, Thompson Sampling and UCB completely flatten: they keep choosing the best arm, not only because they correctly estimated the mean but because there are no more windows after step $t = 2692$, corresponding to the total available data. The fact that this trend is not seen in ϵ -greedy is due to its exploration nature. LinUCB is very similar to UCB and Thompson Sampling, but performs better. What distinguishes it from classic UCB is how exploration is conducted. While UCB exploration can be described as arm-level, that is related to time step t and how many times an arm is pulled, LinUCB explores in the feature space for arm: the exploration bonus, as explained in Section 2.5.2, depends on α , $\mathbf{x}_t$, and the matrix $\mathbf{A}_a^{-1}$, i.e., $\hat{\theta}_a$. In a contextual bandit, the agent explores more consciously, taking into account how features interact with each arm’s history (via the matrix $\mathbf{A}$). The variability in run-to-run is higher in both Thompson Sampling and ϵ -greedy algorithms: it is caused by their stochasticity and by changes in the window, which identify new feasible and best arms. UCB-like algorithms have narrower confidence intervals. Moreover, LinUCB selects the third level very few times: it is due to its nature of exploration based on the context and no longer only on arm-specific quantities.

4.5.1 Best Algorithm

In Table 14 shown in Section 3.5.1, LinUCB was identified as the algorithm with the best behavior, based on the mean and the 95% confidence intervals calculated on the sums of cumulative reward and cumulative regret over the 10 runs. As discussed in static case (Section 4.4.4), the results presented for cumulative regret can be staggered, since it was calculated with 0 penalty when an unfeasible arm is pulled.

4.6. Deployment

Since Thompson Sampling in the aggregated configuration yields the best pair, its Python code is ready to be implemented in the Number Tracing serious game and tested in a real-world context. Having already been trained on real data, it can be implemented using the final distributions obtained from simulation as priors, rather than a uniform one. An example of new priors is shown in Figure 23.

5. Conclusions

In this work, an RL-based system that automatically adjusts the difficulty progression in Number Tracing, a game-based handwriting training, was developed. In particular, several MAB algorithms were investigated to identify the best-performing algorithm and the configuration that best suits the problem. Two approaches were considered: static and dynamic environments. In the first case, Thompson Sampling, using the entire dataset, in an aggregated configuration, results as the most effective, both in terms of cumulative reward and cumulative regret. On the other hand, in a dynamic context, LinUCB performance overcomes the other.

5.1. Limitation of the Current Work

In both static and dynamic cases, the main problem was the limited available data. In a context where it is necessary to develop a system that not only automatically adjusts the difficulty but also accounts for differences among children, it is essential to consider child-specific/cluster configurations rather than an aggregated one. As already mentioned, Thompson Sampling in an aggregated configuration has the highest cumulative reward. These results depend significantly on the sample size used in the static simulation. Indeed, each child has played Number Tracing 92 times on average: as a consequence, the simulation in individual configuration used 92 data points on average compared to the aggregation one, where all the available data were uniformly randomly resampled. For this reason, the aggregated setting gave better results than the child-specific setting. In fact, a larger available data set leads to reduced variance estimates and a better approximation of the data distribution.

The same problem can be seen compared to child-specific with cluster configuration, where better results are obtained. Despite that, while it may be useful to group children with similar characteristics, this information cannot be known a priori; it can only be determined after offline data analysis. Therefore, a limitation is that cluster aggregation can be used only in a subsequent deployment phase, after data have been collected and statistically analyzed.

In dynamic simulation, the τ parameter represented the number of sessions required to reach 63% of the maximum accuracy and was derived from the accuracy data and used as shared context. In addition to having a priori knowledge that it should not have, there is currently no specific function context $\mathbf{x}_t$ modeled on time-in-air. Therefore, the model does not yet accurately reflect the children’s actual behavior.

5.2. Future Developments

Integrating the algorithm into the game would allow automatic adjustment of the difficulty level, maximizing performance and effectively easing the burden on teachers who would otherwise do it manually for an entire, often large, class. A future development could not only maximize performance while accounting for engagement measures, but also encourage players to try new levels, even if they appear less promising to the MAB algorithms. In fact, as already mentioned in the study [28] in Section 1.3, the authors have highlighted the need to focus especially on the learning process, including trying different strategies (difficulty levels) even if the sub-optimal performance from an RL point of view.

In addition, by collecting more data, the preferred configuration - individual one - would likely produce better results. In an educational context, being able to provide personalized training based on the unique characteristics of each child would favor adherence and, even more, involvement. Future work should also include a comparative study between a control group (without the algorithm implementation) and an experimental one with the proposed framework to validate its effectiveness in improving outcomes and participation, as discussed in Section 2.6.

Moreover, the experimental analysis of the dynamic case should continue, implementing a context-specific model $\mathbf{x}_t$ for the engagement measure in LinUCB. A comparative study between static and dynamic cases should be considered to evaluate whether the expansion of the original model could be an effective alternative and more representative of reality. And in conclusion, a further step forward might be to model the improvement in performance across levels even when they are not pulled by the learner. In a real-world case, children enhance their handwriting ability between one gaming session and another.

6. Bibliography and Citations

References

- [1] Tanya Santangelo and Steve Graham. A Comprehensive Meta-analysis of Handwriting Instruction, 6 2016. ISSN 1573336X.
- [2] Stephen M Lange and Brent Thompson. Early identification and interventions for children at risk for learning disabilities. *International Journal of Special Education*, 21(3):108–119, 2006.
- [3] Michael McCloskey and Brenda Rapp. Developmental dysgraphia: An overview and framework for research. *Cognitive neuropsychology*, 34(3-4):65–82, 2017.
- [4] Peter J. Chung, Dilip R. Patel, and Iman Nizami. Disorder of written expression and dysgraphia: definition, diagnosis, and management, 2 2020. ISSN 22244344.
- [5] S. P. Aslam and P. Carugno. Learning disorder, 2025. Available from: <https://www.ncbi.nlm.nih.gov/books/NBK554371/> (accessed January 23, 2026).
- [6] Masne Kadar, Farahiyah Wan Yunus, Eileen Tan, Siaw Chui Chai, Nor Afifi Razaob@ Razab, and Dur-ratul Husna Mohamat Kasim. A systematic review of occupational therapy intervention for handwriting skills in 4–6 year old children. *Australian Occupational Therapy Journal*, 67(1):3–12, 2020.
- [7] Ken Rowe. Effective teaching practices for students with and without learning difficulties: Constructivism as a legitimate theory of learning and of teaching? 2006.

- [8] Daniel J Reschly. Learning disabilities identification: Primary intervention, secondary intervention, and then what? *Journal of learning Disabilities*, 38(6):510–515, 2005.
- [9] Suatmi Murnani, Kazuharu Hashitsume, and Tomohiko Igasaki. Handwriting difficulties: A review of recent advances in the identification and intervention. *IEEE Access*, 2025.
- [10] Maria Luisa Lorusso, Mirta Vernice, Marina Dieterich, Daniela Brizzolara, Enrica Mariani, Salvatore De Masi, Franca D’Angelo, Eleonora Lacorte, and Alfonso Mele. The process and criteria for diagnosing specific learning disorders: Indications from the consensus conference promoted by the italian national institute of health. *Annali dell’Istituto superiore di sanita*, 50:77–89, 2014.
- [11] Repubblica Italiana. Legge 8 ottobre 2010, n. 170. nuove norme in materia di disturbi specifici di apprendimento in ambito scolastico [law october 8th, 2010, n. 170. new rules on specific learning disabilities in schools], 10 2010. URL <https://www.gazzettaufficiale.it/eli/id/2010/10/18/010G0192/sg>. Published in Gazzetta Ufficiale n. 244, October 18th, 2010.
- [12] Italian Ministry of Health. Guideline on the management of specific learning disabilities (linea guida sulla gestione dei disturbi specifici dell’apprendimento), 2022.
- [13] Silja Litvin, Rob Saunders, Markus A Maier, and Stefan Lüttke. Gamification as an approach to improve resilience and reduce attrition in mobile mental health interventions: a randomized controlled trial. *PloS one*, 15(9):e0237220, 2020.
- [14] Tarja Susi, Mikael Johannesson, and Per Backlund. Serious games: An overview. 2007.
- [15] Diego Ávila-Pesántez, Luis A Rivera, and Mayra S Alban. Approaches for serious game design: A systematic literature review. *Computers in education journal*, 8(3), 2017.
- [16] Shuja Ud Din, Muhammad Zeeshan Baig, and Muhammad Khateeb Khan. Serious Games: An Updated Systematic Literature Review. *arXiv preprint arXiv:2306.03098*, 5 2023. URL <http://arxiv.org/abs/2306.03098>.
- [17] Christo Dichev and Darina Dicheva. Gamifying education: what is known, what is believed and what remains uncertain: a critical review. *International journal of educational technology in higher education*, 14(1):9, 2017.
- [18] Yu Zhonggen. A Meta-Analysis of Use of Serious Games in Education over a Decade. *International Journal of Computer Games Technology*, 2019(1), 2019. doi: <https://doi.org/10.1155/2019/4797032>. URL <https://onlinelibrary.wiley.com/doi/abs/10.1155/2019/4797032>.
- [19] Theofylaktos Anastasiadis, Georgios Lampropoulos, and Kerstin Siakas. Digital game-based learning and serious games in education. *International Journal of Advances in Scientific Research and Engineering*, 4 (12):139–144, 2018.
- [20] Houda Mouaheb, Ahmed Fahli, Mohammed Moussetad, and Said Eljamali. The Serious Game: What Educational Benefits? *Procedia - Social and Behavioral Sciences*, 46:5502–5508, 2012. ISSN 18770428. doi: 10.1016/j.sbspro.2012.06.465.
- [21] Michael F Young, Stephen Slota, Andrew B Cutter, Gerard Jalette, Greg Mullin, Benedict Lai, Zeus Simeoni, Matthew Tran, and Mariya Yukhymenko. Our princess is in another castle: A review of trends in serious gaming for education. *Review of educational research*, 82(1):61–89, 2012.
- [22] Maosen Xu, Yong Luo, Yu Zhang, Ruolan Xia, Hong Qian, and Xiuhe Zou. Game-based learning in medical education. *Frontiers in public health*, 11:1113682, 2023.
- [23] Manuel Arias-Calderón, Javiera Castro, and Silvina Gayol. Serious games as a method for enhancing learning engagement: Student perception on online higher education during covid-19. *Frontiers in Psychology*, 13:889975, 2022.
- [24] Jeanine Krath, Linda Schürmann, and Harald FO Von Korflesch. Revealing the theoretical basis of gamification: A systematic review and analysis of theory in research on gamification, serious games and game-based learning. *Computers in human behavior*, 125:106963, 2021.
- [25] Sandra Sampayo-Vargas, Chris J Cope, Zhen He, and Graeme J Byrne. The effectiveness of adaptive difficulty adjustments on students’ motivation and learning in an educational computer game. *Computers & Education*, 69:452–462, 2013.

- [26] Wang Qiang and Zhan Zhongli. Reinforcement learning model, algorithms and its application. In *2011 International Conference on Mechatronic Science, Electric Engineering and Computer (MEC)*, pages 1143–1146. IEEE, 2011.
- [27] Aleksandrs Slivkins. Introduction to Multi-Armed Bandits. *Foundations and Trends® in Machine Learning*, 4 2024. URL <http://arxiv.org/abs/1904.07272>.
- [28] Bahar Memarian and Tenzin Doleck. A scoping review of reinforcement learning in education. *Computers and Education Open*, 6, 6 2024. ISSN 26665573. doi: 10.1016/j.caeo.2024.100175.
- [29] Francesco Bellotti, Riccardo Berta, Alessandro De Gloria, and Ludovica Primavera. Adaptive experience engine for serious games. *IEEE Transactions on Computational Intelligence and AI in Games*, 1(4):264–280, 12 2009. ISSN 1943068X. doi: 10.1109/TCIAIG.2009.2035923.
- [30] Laura Climent, Alessio Longhi, Alejandro Arbelaez, and Maurizio Mancini. A framework for designing Reinforcement Learning agents with Dynamic Difficulty Adjustment in single-player action video games. *Entertainment Computing*, 50, 5 2024. ISSN 18759521. doi: 10.1016/j.entcom.2024.100686.
- [31] Su Xue, Meng Wu, John Kolen, Navid Aghdaie, and Kazi A Zaman. Dynamic Difficulty Adjustment for Maximized Engagement in Digital Games. In *Proceedings of the 26th International Conference on World Wide Web Companion*, WWW '17 Companion, pages 465–471, Republic and Canton of Geneva, CHE, 2017. International World Wide Web Conferences Steering Committee. ISBN 9781450349147. doi: 10.1145/3041021.3054170. URL <https://doi.org/10.1145/3041021.3054170>.
- [32] Xizhi Zhan. A Review of Multi-Armed Bandit Algorithms in Player Modeling and Game Design. *ITM Web of Conferences*, 73:02016, 2025. doi: 10.1051/itmconf/20257302016.
- [33] Anton Vinogradov and Brent Harrison. Using multi-armed bandits to dynamically update player models in an experience managed environment. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, volume 18, pages 207–214, 2022.
- [34] Zahra Amiri and Yoonas A. Sekhavat. Intelligent Adjustment of Game Properties at Run Time Using Multi-armed Bandits. *The Computer Games Journal*, 8(3-4):143–156, 12 2019. doi: 10.1007/s40869-019-00083-3.
- [35] Jackson A. Killian, Manish Jain, Yugang Jia, Jonathan Amar, Erich Huang, and Milind Tambe. New Approach to Equitable Intervention Planning to Improve Engagement and Outcomes in a Digital Health Program: Simulation Study. *JMIR Diabetes*, 9, 2024. ISSN 23714379. doi: 10.2196/52688.
- [36] Elliot Lee, Mariel S Lavieri, and Michael Volk. Optimal screening for hepatocellular carcinoma: A restless bandit model. *Manufacturing & Service Operations Management*, 21(1):198–212, 2019.
- [37] Aldo Pacchiano, Mohammad Ghavamzadeh, Peter Bartlett, and Heinrich Jiang. Stochastic bandits with linear constraints. In *International conference on artificial intelligence and statistics*, pages 2827–2835. PMLR, 2021.
- [38] Aldo Pacchiano, Mohammad Ghavamzadeh, and Peter Bartlett. Contextual bandits with stage-wise constraints. *Journal of machine learning research*, 26(170):1–57, 2025.
- [39] Tor Lattimore and Csaba Szepesvári. *Bandit algorithms*. Cambridge University Press, 2020.
- [40] J Russo Daniel, Van Roy Benjamin, Kazerouni Abbas, Osband Ian, and Wen Zheng. A tutorial on thompson sampling. *Foundations and trends® in machine learning*, 11(1):1–99, 2018.
- [41] Aditya Gangrade and Venkatesh Saligrama. Constrained Linear Thompson Sampling. 6 2025. URL <http://arxiv.org/abs/2503.02043>.
- [42] Lihong Li, Wei Chu, John Langford, and Robert E. Schapire. A Contextual-Bandit Approach to Personalized News Article Recommendation, 3 2012. URL <http://arxiv.org/abs/1003.0146><http://dx.doi.org/10.1145/1772690.1772758>.

Abstract in lingua italiana

La scrittura rappresenta una competenza essenziale e la tecnologia può rivelarsi uno strumento fondamentale per supportare studenti con difficoltà. In particolare, i serious game possono offrire un sistema che permette di esercitarsi in modo divertente, evitando noia e frustrazione spesso associati a esercizi di potenziamento ripetitivi. La presente tesi propone un framework di Multi-Armed Bandit (MAB) in grado di adattare in tempo reale il livello di difficoltà in base alle prestazioni del giocatore mantenendo un'elevata partecipazione nel gioco "Number Tracing", un serious game per tablet dedicato alla scrittura, giocato utilizzando uno stilo. Il gioco è stato modellizzato come un problema di MAB vincolato, in cui ogni livello rappresenta un "arm" e viene imposto un vincolo minimo sul livello di coinvolgimento. La ricompensa è stata definita come l'accuratezza del tracciato, misurata come la percentuale del tracciato entro i bordi, mentre il coinvolgimento è stato definito attraverso il tempo in aria, che cattura la partecipazione attiva del giocatore. Nel setup sperimentale, è stato assunto un modello statico dei giocatori. Sono stati valutati diversi algoritmi di MAB, tra cui epsilon-greedy, Upper Confidence Bound e Thompson Sampling, in differenti configurazioni di utilizzo, e testati su dati raccolti durante un protocollo di potenziamento di tre mesi su 66 bambini di prima elementare. I risultati mostrano che Thompson Sampling, su dati aggregati, ottiene le migliori prestazioni in termini di ricompensa e regret cumulativi. Poiché l'ipotesi di staticità rappresenta una forte semplificazione della realtà, è stato successivamente sviluppato un modello dinamico, considerando una sola configurazione operativa, includendo anche l'algoritmo contestuale Linear UCB, che ha mostrato le migliori prestazioni. I risultati ottenuti evidenziano il potenziale dei framework proposti, seppur sottolineando la necessità di ulteriori studi su dataset più ampi e di un'analisi riguardo l'efficacia in un contesto reale.

Parole chiave: Multi-Armed Bandit vincolato, bandit contestuale vincolato, serious game, bambini, scrittura, potenziamento